PHYB57 · CHAPTER 16 · 第 16 章

最小二乘回归 · 全面复习讲义

Least Squares Regression · Comprehensive Review Notes

中英双语 · KaTeX 公式 · 深色阅读版

Bilingual · KaTeX formulas · dark reading edition

KaTeX 公式 · KaTeX　中英对照 · Bilingual　每页考点速记 · Key points on every page　两份课件合订 · C16_1 + C16_2　按幻灯片顺序 · Slide order　拓展 + 6 道练习 · Extensions + 6 problems

合并 C16_1（10 页）和 C16_2（6 页）两份幻灯片，按原顺序讲完：问题定义与动机、估计函数与基函数、弹簧例子、残差与总平方误差、线性代数推导（投影与正规方程）、多元微积分推导、Python 的四种解法（inv、pinv、lstsq、curve_fit）、对数技巧、多项式回归与过拟合、用 curve_fit 直接做非线性拟合。

> Merges the two slide decks C16_1 (10 slides) and C16_2 (6 slides), in their original order: problem and motivation, estimation and basis functions, the spring example, residuals and total squared error, the linear-algebra derivation (projection, normal equation), the calculus derivation, four Python methods (inv, pinv, lstsq, curve_fit), log tricks, polynomial regression and overfitting, and direct nonlinear fitting with curve_fit.

另外（标 拓展 EXT）：直线的闭式解、投影的证明、手算例题、为什么用 lstsq 而不是求逆（条件数）、对数技巧和直接拟合为什么结果不同、阶数与过拟合、方法选择表；最后是公式总表和 6 道带解答的练习。所有数值都用 Python（NumPy 2.5、SciPy 1.18）复算；幻灯片上的代码在这些版本上逐段跑通。

> Added (tagged 拓展 EXT): the closed-form line fit, a proof of the projection argument, a worked example by hand, why lstsq beats explicit inversion (condition numbers), why the log trick and the direct fit disagree, order and overfitting, and a method-selection table; then a formula sheet and 6 worked problems. Every number was recomputed in Python (NumPy 2.5, SciPy 1.18), and every code cell on the slides was run on these versions.

PHYB57 复习用 · 2026 年 11 月 5 日

> For PHYB57 review · 5 November 2026

考点速记 KEY POINTS

- 目标：$\min_\beta \|Y - A\beta\|_2^2$。
  Goal: $\min_\beta \|Y - A\beta\|_2^2$.
- "线性"指对参数线性，不是对 x 线性。
  "Linear" means linear in the parameters, not in x.
- 正规方程：$A^TA\beta = A^TY$。
  Normal equation: $A^TA\beta = A^TY$.
- 指数、幂函数：取 $\ln$ 线性化，或用 curve_fit。
  Exponential, power: linearise with $\ln$, or use curve_fit.

本章地图与记号

Chapter map and notation

节 Sec.	内容 Content	幻灯片 Slides
1	什么是最小二乘回归 · 两种推导 · Python 概览 What it is · two derivations · Python overview	C16_1: 1–3
2	估计函数与基函数 · 弹簧例子 · 残差与总平方误差 Estimation and basis functions · spring example · residuals and E	C16_1: 4–6
3	线性代数推导：投影与正规方程 +证明 proof Linear-algebra derivation: projection and normal equation	C16_1: 7–8
4	多元微积分推导 +手算例题 worked example Multivariable-calculus derivation	C16_1: 9–10
5	Python：inv、pinv、lstsq、curve_fit +条件数 conditioning Python methods for least squares	C16_2: 1–2
6	非线性函数：对数技巧、多项式回归、curve_fit +对比 comparison Nonlinear functions: log tricks, polynomials, curve_fit	C16_2: 3–6
—	公式总表 · 练习 Q1–Q6 · 答案与来源 Formula sheet · problems · key and sources	—

记号 / Notation

数据与模型 / data and model

$$(x_i, y_i),\ i = 1, \dots, m; \qquad \hat{y}(x) = \sum_{j=1}^{n} \alpha_j f_j(x)$$

矩阵与向量 / matrix and vectors

$$A_{ij} = f_j(x_i),\ A \in \mathbb{R}^{m \times n}; \quad \beta_j = \alpha_j$$

预测与残差 / prediction and residual

$$\hat{Y} = A\beta, \qquad e = Y - \hat{Y}$$

总平方误差 / total squared error

$$E = \|e\|_2^2 = \sum_{i=1}^{m} e_i^2$$

同一个字母，不同的意思 / One letter, different meanings

- m：C16_1 第 9 页是数据点个数；第 2 页 $y = mx + b$ 里是斜率；C16_2 第 4 页 $y = bx^m$ 里是幂指数。
  > m: number of data points on C16_1 slide 9; the slope in $y = mx + b$ on slide 2; the exponent in $y = bx^m$ on C16_2 slide 4.
- β：C16_1 第 7–8 页是参数列向量；C16_2 第 4、6 页 $y = \alpha e^{\beta x}$ 里是指数中的系数。
  > β: the parameter vector on C16_1 slides 7–8; the rate in $y = \alpha e^{\beta x}$ on C16_2 slides 4 and 6.
- 代码里的变量 `alpha` 装的是整个参数向量（斜率和截距），不是单个 α。
  > The code variable `alpha` holds the whole parameter vector (slope and intercept), not a single α.
- 标 拓展 EXT 的内容超出幻灯片。
  > Items tagged 拓展 EXT go beyond the slides.

考点速记 KEY POINTS

- m 个点、n 个参数，通常 $m > n$（超定）。
  m points, n parameters, usually $m > n$ (overdetermined).
- 残差 $e = Y - A\beta$，误差 $E = \|e\|_2^2$。
  Residual $e = Y - A\beta$, error $E = \|e\|_2^2$.
- A 的第 j 列是 f_j 在所有 x_i 的值。
  Column j of A is f_j at every x_i.
- 看清 m、β 在每页指什么。
  Check what m and β mean on each slide.

1.1 什么是最小二乘回归

1.1 What is least squares regression?

定义 / Definition

最小二乘回归是一种数学方法：在一组数据点中找出拟合得最好的直线（或模型），做法是让观测值与预测值之差的平方和最小。科学和工程问题里常用它从实验数据估计未知参数。

> A mathematical method that finds the best-fitting line (or model) through a set of data points by minimising the sum of the squares of the differences between observed data and predicted values. It is commonly used in science and engineering to estimate unknown parameters from experimental data.

动机 / Motivation

实际问题中，模型参数（例如一组弹簧实验中的弹簧刚度）不能直接知道，只能依靠通常带有噪声的实验数据。即使数据有误差或噪声，最小二乘也能给出最可能的模型参数。

> In practice, model parameters (such as a spring stiffness in a set of spring experiments) are not known directly; we rely on experimental data that usually contains noise. Least squares finds the most likely model parameters even when the data contains errors or noise.

例子：弹簧刚度 / Example: spring stiffness

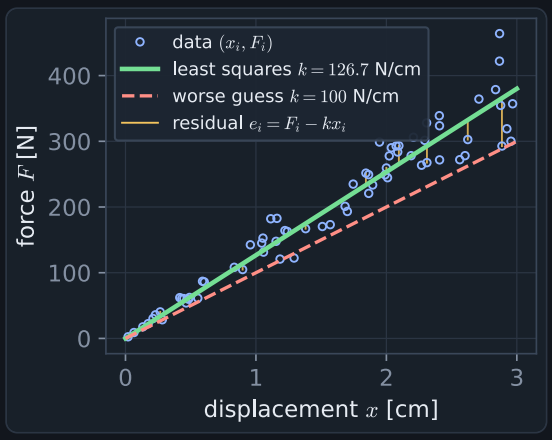
由力 F 和位移 x 的实验数据，确定线性弹簧的刚度 k：

> Determine the stiffness k of a linear spring from experimental data of force F and displacement x:

$$F = kx$$

右图按幻灯片的例子重画：圆圈是带噪声的测量；绿线是最小二乘给出的 k；红虚线是一个较差的猜测；黄色竖线是残差 $e_i = F_i - kx_i$。最小二乘在所有直线中选出残差平方和最小的那一条。

> The figure redraws the slide's example: circles are noisy measurements, the green line is the least-squares k, the red dashed line is a worse guess, and yellow segments are residuals $e_i = F_i - kx_i$. Least squares picks the line whose squared residuals sum to the smallest value.



重画图（数据按 $k = 130$ 加噪声生成）

/ redrawn; data generated with $k = 130$ plus noise

编者注 / Editorial note

幻灯片原图（第 1、5、6、10 页重复出现）有一条绿线和一条红线，没有图例；上面的颜色说明是本讲义加的。原图的散点越往右越分散，说明噪声随位移变大。

> The slide figure (repeated on slides 1, 5, 6 and 10) shows a green and a red line without a legend; the colour legend above is added here. Its scatter widens to the right, so the noise grows with displacement.

考点速记 KEY POINTS

- 最小二乘：使 $\sum(y_i - \hat{y}_i)^2$ 最小。
  Least squares: minimise $\sum(y_i - \hat{y}_i)^2$.
- 弹簧：$F = kx$，待估参数是 k。
  Spring: $F = kx$, the parameter is k.
- 用途：从带噪声数据估计参数。
  Use: estimate parameters from noisy data.
- 残差是 y 方向的竖直距离，不是到直线的垂直距离。
  Residuals are vertical (y) distances, not perpendicular ones.

1.2 推导最小二乘的两种方法

1.2 Methods of deriving least squares regression

两种关键推导 / Two key derivations

- **线性代数方法**：根据数据构造一个线性方程组，并让残差平方和最小（第 3 节）。

  > **Linear algebra approach**: build a system of linear equations from the data and minimise the residual sum of squares (Section 3).

- **多元微积分方法**：对残差平方和求导并令导数为零，得到最优参数（第 4 节）。

  > **Multivariable calculus approach**: differentiate the residual sum of squares and set the derivatives to zero to find the optimal parameters (Section 4).

两种方法得到同一个公式：正规方程 $A^TA\beta = A^TY$。

> Both reach the same formula, the normal equation $A^TA\beta = A^TY$.

线性模型例子 / Linear model example

对线性关系 $y = mx + b$，目标是让误差 E 最小，其中

> For a linear relationship $y = mx + b$, the goal is to minimise the error E, where

$$E = \sum \left(y_{\text{observed}} - y_{\text{predicted}}\right)^2 = \sum_{i=1}^{N} \left(y_i - (mx_i + b)\right)^2$$

这里 m 是斜率，所以点数记作 N / m is the slope here, so the point count is N

解出来就得到最优斜率 m 和截距 b。

> The solution yields the optimal slope m and intercept b.

直线的闭式解 拓展 EXT / Closed form for a line

$$\frac{\partial E}{\partial b} = -2\sum_i \left(y_i - mx_i - b\right) = 0 \quad \Rightarrow \quad \bar{y} = m\bar{x} + b$$

$$\frac{\partial E}{\partial m} = -2\sum_i x_i\left(y_i - mx_i - b\right) = 0 \quad \Rightarrow \quad \sum x_iy_i = m\sum x_i^2 + b\sum x_i$$

$$m = \frac{N\sum x_iy_i - \sum x_i \sum y_i}{N\sum x_i^2 - \left(\sum x_i\right)^2}, \qquad b = \bar{y} - m\bar{x}$$

$\bar{x}, \bar{y}$ 是平均值；最佳直线一定经过 $(\bar{x}, \bar{y})$ / $\bar{x}, \bar{y}$ are the means; the best line passes through $(\bar{x}, \bar{y})$

考试提醒 / Exam reminder

模型带截距 b 时，$\partial E/\partial b = 0$ 直接给出 $\sum e_i = 0$：残差之和为零。没有截距的模型（如 $F = kx$）一般不满足这一条，见 Q1。

> With an intercept b, $\partial E/\partial b = 0$ gives $\sum e_i = 0$: the residuals sum to zero. A model without an intercept (such as $F = kx$) generally does not, see Q1.

考点速记 KEY POINTS

- 代数法与微积分法得到同一正规方程。
  Algebra and calculus give the same normal equation.
- 有截距 ⇒ $\sum e_i = 0$。
  Intercept present ⇒ $\sum e_i = 0$.
- $b = \bar{y} - m\bar{x}$：直线过均值点。
  $b = \bar{y} - m\bar{x}$: the line passes through the mean point.
- 分母 $N\sum x_i^2 - (\sum x_i)^2 = 0$ 当且仅当所有 x_i 相同。
  The denominator is 0 only if all x_i are equal.

1.3 Python 实现与非线性扩展（总览）

1.3 Least squares in Python and nonlinear extensions

在 Python 中实现 / Implementing in Python

- **NumPy** 和 **SciPy** 提供了高效的最小二乘函数。
  > Libraries like **NumPy** and **SciPy** provide efficient functions for least squares regression.
- 例如：用 `numpy.linalg.lstsq()` 求解线性方程组，用 `scipy.optimize.curve_fit()` 拟合非线性模型。
  > Example: `numpy.linalg.lstsq()` to solve a linear system, or `scipy.optimize.curve_fit()` for nonlinear models.

非线性回归 / Nonlinear regression

- 最小二乘也能推广到非线性函数，例如指数、对数或多项式模型。
  > Least squares extends to nonlinear functions, such as exponential, logarithmic or polynomial models.
- 做法是把问题线性化，或者用迭代优化方法让复杂模型的误差最小。
  > The process linearises the problem or uses iterative optimisation to minimise the error of complex models.

本章用到的工具 拓展 EXT / Tools used in this chapter

调用 Call	解决什么 What it solves	节 Sec.
`np.linalg.inv(A.T@A)@A.T@y`	直接套正规方程（教学用）· the normal equation directly (teaching)	5.1
`np.linalg.pinv(A)@y`	伪逆 $(A^TA)^{-1}A^T$ · the pseudoinverse	5.3
`np.linalg.lstsq(A, y, rcond=None)`	线性最小二乘，基于 SVD（推荐）· linear LS via SVD (recommended)	5.3, 5.4
`np.polyfit(x, y, n)`	n 阶多项式回归 · degree-n polynomial regression	6.4
`optimize.curve_fit(f, x, y)`	任意模型，迭代求解 · any model, solved iteratively	5.3, 6.6

编者注 / Editorial note

幻灯片把多项式模型列在"非线性"里。多项式对 x 是非线性的，但对系数 $a_0, \ldots, a_n$ 是线性的，所以用普通的线性最小二乘一步解出，不需要迭代（第 6.4 节）。需要迭代的是参数出现在非线性位置的模型，例如 $\alpha e^{\beta x}$ 里的 β。

> The slide lists polynomial models under "nonlinear". A polynomial is nonlinear in x but linear in its coefficients $a_0, \ldots, a_n$, so ordinary linear least squares solves it in one step, with no iteration (Section 6.4). Iteration is needed when a parameter sits in a nonlinear position, such as β in $\alpha e^{\beta x}$.

考点速记 KEY POINTS

- 线性模型：`lstsq`；非线性：`curve_fit`。
  Linear models: `lstsq`; nonlinear: `curve_fit`.
- 非线性两条路：线性化，或迭代优化。
  Nonlinear: linearise, or iterate.
- 多项式回归仍是线性最小二乘。
  Polynomial regression is still linear least squares.
- 判断依据：参数是否以线性方式出现。
  Test: do the parameters enter linearly?

2.1 估计函数与基函数

2.1 Estimation function and basis functions

定义：最小二乘回归要找一个估计函数 $\hat{y}(x)$，使它最好地拟合数据点 (x_i, y_i)。估计函数通常是"基函数"（basis functions）的线性组合：

> **Definition**: least squares regression finds an estimation function $\hat{y}(x)$ that best fits the data points (x_i, y_i). The estimation function is typically a linear combination of basis functions:

$$\hat{y}(x) = \sum_{i=1}^{n} \alpha_i f_i(x)$$

- α_i 是待定参数。
  > α_i are the parameters to be determined.
- $f_i(x)$ 是基函数，必须**线性无关**：任何一个都不能写成其他几个的线性组合。
  > $f_i(x)$ are the basis functions, which must be **linearly independent**: none can be a linear combination of the others.

在每个数据点写出来 / Written out at every data point

$$\begin{aligned}
\hat{y}(x_1) &= \alpha_1 f_1(x_1) + \alpha_2 f_2(x_1) + \cdots + \alpha_n f_n(x_1), \\
\hat{y}(x_2) &= \alpha_1 f_1(x_2) + \alpha_2 f_2(x_2) + \cdots + \alpha_n f_n(x_2), \\
&\vdots \\
\hat{y}(x_m) &= \alpha_1 f_1(x_m) + \alpha_2 f_2(x_m) + \cdots + \alpha_n f_n(x_m).
\end{aligned}$$

m 个方程、n 个未知数；系数排成 $m \times n$ 矩阵 $A_{ij} = f_j(x_i)$（幻灯片标的"m x n matrix"就是它）

/ m equations, n unknowns; the coefficients form the $m \times n$ matrix $A_{ij} = f_j(x_i)$ (the slide's "m x n matrix")

目标 / Objective

让估计值与实际数据之间的总平方误差最小。平方误差 E 定义为对所有数据点求和：

> Minimise the total squared error between the estimated and actual data points. The squared error E is a sum over the data points:

$$E = \sum_{i=1}^{m} \left(y_i - \hat{y}(x_i)\right)^2$$

y_i：实际的因变量；$\hat{y}(x_i)$：模型在 x_i 处的估计值 / y_i: actual dependent variable; $\hat{y}(x_i)$: model estimate at x_i

"线性"是对参数而言 拓展 EXT / "Linear" refers to the parameters

f_j 可以是任何已知函数，如 $1, x, x^2, \sin x, e^x$。$\hat{y} = \alpha_1 + \alpha_2 x + \alpha_3 \sin x$ 仍是线性最小二乘；$\hat{y} = \alpha e^{\beta x}$ 不是，因为 β 在指数里。若取 $f_1 = x$，$f_2 = 2x$，A 的两列成比例，$A^T A$ 不可逆，参数无法唯一确定。

> f_j can be any known function, e.g. $1, x, x^2, \sin x, e^x$. $\hat{y} = \alpha_1 + \alpha_2 x + \alpha_3 \sin x$ is still linear least squares; $\hat{y} = \alpha e^{\beta x}$ is not, because β sits in the exponent. With $f_1 = x$, $f_2 = 2x$ the two columns of A are proportional, $A^T A$ is singular and the parameters are not unique.

考点速记 KEY POINTS

- $\hat{y}(x) = \sum_j \alpha_j f_j(x)$，对 α 线性。
  $\hat{y}(x) = \sum_j \alpha_j f_j(x)$, linear in α.
- 基函数线性无关 ⇔ A 列满秩。
  Independent basis ⇔ A has full column rank.
- A 是 $m \times n$，$A_{ij} = f_j(x_i)$。
  A is $m \times n$, $A_{ij} = f_j(x_i)$.
- $E = \sum_{i=1}^{m} (y_i - \hat{y}(x_i))^2$。
  $E = \sum_{i=1}^{m} (y_i - \hat{y}(x_i))^2$.

2.2 弹簧系统例子

2.2 Spring system example

为了更好地理解这个概念，考虑线性弹簧的力–位移关系。

To understand the concept better, consider the force–displacement relationship of a linear spring.

胡克定律 / Hooke's law

$$F(x) = kx$$

力 F 与位移 x 成正比；k 是刚度，即模型参数

/ force F vs displacement x; stiffness k is the model parameter

估计函数 / estimation function

$$\hat{y}(x) = kx$$

唯一的基函数 $f(x) = x$，待定参数是 k

/ the only basis function is $f(x) = x$; the parameter is k

目标：让平方误差最小，找出最符合力–位移数据的 k。

Goal: minimise the squared error to find the value of k that best fits the force–displacement data.

一个参数的闭式解 拓展 EXT / Closed form with one parameter

$$E(k) = \sum_{i=1}^{m} (F_i - kx_i)^2, \qquad \frac{\mathrm{d}E}{\mathrm{d}k} = -2\sum_{i=1}^{m} x_i\,(F_i - kx_i) = 0$$

$$k = \frac{\sum_i x_i F_i}{\sum_i x_i^2}$$

A 只有一列 $(x_1, \ldots, x_m)^T$，所以 $A^TA = \sum x_i^2$，$A^TY = \sum x_i F_i$

/ A has one column $(x_1, \ldots, x_m)^T$, so $A^TA = \sum x_i^2$, $A^TY = \sum x_i F_i$

$E(k)$ 是开口向上的抛物线（$\mathrm{d}^2E/\mathrm{d}k^2 = 2\sum x_i^2 > 0$），所以这个驻点就是最小值。第 1.1 页的重画图用这个公式算出 $k = 126.7\ \mathrm{N/cm}$（生成数据时用的是 130）。

$E(k)$ is an upward parabola ($\mathrm{d}^2E/\mathrm{d}k^2 = 2\sum x_i^2 > 0$), so this stationary point is the minimum. For the redrawn figure on page 1.1 the formula gives $k = 126.7\ \mathrm{N/cm}$ (the data were generated with 130).

注意 / Caution

过原点模型没有截距，所以残差之和 $\sum e_i$ 一般**不为零**；成立的只有 $\sum x_i e_i = 0$（就是上面令导数为零的那一式）。把 $F = kx$ 换成 $F = kx + b$ 会得到不同的 k（Q1 里 50.03 变成 49.2）。

A through-origin model has no intercept, so $\sum e_i$ is generally **not zero**; only $\sum x_i e_i = 0$ holds (the zero-derivative condition above). Switching from $F = kx$ to $F = kx + b$ changes k (in Q1, 50.03 becomes 49.2).

考试提醒 / Exam reminder

选模型要看物理：弹簧在 $x = 0$ 时 $F = 0$，所以 $F = kx$ 比 $F = kx + b$ 更合理。单位：x 用 cm、F 用 N，则 k 的单位是 $\mathrm{N/cm}$。

Choose the model from the physics: a spring has $F = 0$ at $x = 0$, so $F = kx$ is more sensible than $F = kx + b$. Units: with x in cm and F in N, k is in $\mathrm{N/cm}$.

考点速记 KEY POINTS

- $\hat{y} = kx$：一个基函数 $f(x) = x$。
  $\hat{y} = kx$: one basis function $f(x) = x$.
- 无截距：$\sum x_i e_i = 0$，但 $\sum e_i \neq 0$。
  No intercept: $\sum x_i e_i = 0$ but $\sum e_i \neq 0$.
- $k = \sum x_i F_i / \sum x_i^2$。
  $k = \sum x_i F_i / \sum x_i^2$.
- k 的单位 = F 的单位 / x 的单位。
  Units of k = units of F / units of x.

2.3 让总平方误差最小

2.3 Minimising the total squared error

误差的定义 / Error definition

每个数据点的误差（残差，residual）定义为：

> The individual errors (residuals) are defined as:

$$e_i = y_i - \hat{y}(x_i)$$

目标是让平方误差 E 最小，它就是 m 维残差向量 e 的 L2 范数的平方：

> The objective is to minimise the squared error E, the squared L2 norm of the m-dimensional residual vector e:

向量形式 / vector form

$$E = \|e\|_2^2$$

求和形式 / sum form

$$E = \sum_{i=1}^{m} \left(y_i - \hat{y}(x_i)\right)^2$$

编者注 / Editorial note

幻灯片说 E"本质上是 e 的 (L2) 范数"。准确地说是范数的**平方**：$\|e\|_2 = \sqrt{\sum e_i^2}$，$E = \|e\|_2^2$。幻灯片上的公式写对了。两者在同一组参数处取最小值，所以结论不变。

> The slide calls E "essentially the (L2) norm" of e. Strictly it is the **square** of the norm: $\|e\|_2 = \sqrt{\sum e_i^2}$ and $E = \|e\|_2^2$. The slide's formulas are correct. Both are minimised by the same parameters, so the conclusion is unchanged.

为什么用平方 拓展 EXT / Why squares?

- 平方处处可导，E 是参数的二次函数；令导数为零得到**线性**方程组（第 4 节），可以一步解出。
  > Squares are differentiable everywhere and E is quadratic in the parameters; setting the derivatives to zero gives a **linear** system (Section 4), solved in one step.
- 正负误差不会相互抵消（直接求 $\sum e_i$ 会抵消）。
  > Positive and negative errors cannot cancel (a plain $\sum e_i$ would).
- 大误差被平方放大，所以最小二乘对离群点（outliers）敏感：一个误差为 10 的点贡献 100，相当于 100 个误差为 1 的点。
  > Large errors are amplified, so least squares is sensitive to outliers: one error of 10 contributes 100, as much as 100 errors of 1.
- 若噪声独立、同方差、服从高斯分布，最小二乘估计就是最大似然估计。
  > If the noise is independent, of equal variance and Gaussian, the least-squares estimate is the maximum-likelihood estimate.
- 改用 $\sum |e_i|$（L1）更抗离群点，但没有闭式解，要迭代求解。
  > Using $\sum |e_i|$ (L1) resists outliers better but has no closed form and must be solved iteratively.

考点速记 KEY POINTS

- $e_i = y_i - \hat{y}(x_i)$：观测减预测。
  $e_i = y_i - \hat{y}(x_i)$: observed minus predicted.
- $E = \|e\|_2^2$ 是范数的平方。
  $E = \|e\|_2^2$ is the squared norm.
- 平方 ⇒ 二次函数 ⇒ 线性方程组。
  Squares ⇒ quadratic ⇒ linear system.
- 缺点：对离群点敏感。
  Drawback: sensitive to outliers.

3.1 代数方法：目标与记号

3.1 Algebra approach: objective and notation

目标 / Objective

让观测值与估计值之间的总平方误差最小，从而为一组数据点找到最佳拟合直线。

> Find the best-fit line for a set of data points by minimising the total squared error between the observed and estimated values.

关键概念 / Key concepts

- **数据点**：记作 (x_i, y_i)，x_i 是自变量，y_i 是因变量。
  > **Data points**: (x_i, y_i), where x_i is the independent and y_i the dependent variable.
- **目标**：用线性模型 $\hat{Y} = A\beta$ 估计 x_i 与 y_i 的关系；A 是预测函数（基函数）构成的矩阵，β 是系数 α 组成的向量。
  > **Goal**: estimate the relationship between x_i and y_i with a linear model $\hat{Y} = A\beta$, where A is the matrix of predictor (basis) functions and β is the vector of α coefficients.

预测向量 / prediction vector	参数向量 / parameter vector	矩阵形式 / matrix form
$\hat{Y}_i = \hat{y}(x_i)$	$\beta_i = \alpha_i$	$\hat{Y} = A\beta$

总平方误差：让下式最小，其中 $\hat{Y}$ 是回归模型的预测值：

> **Total squared error**: minimise the following, where $\hat{Y}$ is the predicted value from the regression model:

$$E = \|Y - \hat{Y}\|^2$$

例：直线写成矩阵 拓展 EXT / Example: a line in matrix form

$$\hat{y} = \alpha_1 x + \alpha_2 : \qquad A = \begin{pmatrix} x_1 & 1 \\ x_2 & 1 \\ \vdots & \vdots \\ x_m & 1 \end{pmatrix}, \quad \beta = \begin{pmatrix} \alpha_1 \\ \alpha_2 \end{pmatrix}, \quad A\beta = \begin{pmatrix} \alpha_1 x_1 + \alpha_2 \\ \vdots \\ \alpha_1 x_m + \alpha_2 \end{pmatrix}$$

第 5 节代码 `np.vstack([x, np.ones(len(x))]).T` 造的就是这个 A

/ the code `np.vstack([x, np.ones(len(x))]).T` in Section 5 builds exactly this A

为什么不直接解 $A\beta = Y$ / Why not just solve $A\beta = Y$?

$m > n$ 时（数据点多于参数），方程组 $A\beta = Y$ 是超定的，带噪声的数据一般没有精确解。于是把"解方程"换成"让 $\|Y - A\beta\|$ 最小"。A 不是方阵，也不能求 A^{-1}。

> With $m > n$ (more points than parameters), $A\beta = Y$ is overdetermined and noisy data generally has no exact solution. So "solve the system" becomes "minimise $\|Y - A\beta\|$". A is not square, so A^{-1} does not exist either.

考点速记 KEY POINTS

- $\hat{Y} = A\beta$，β 就是 α 的列向量。
  $\hat{Y} = A\beta$, β is the column of α's.
- 超定 $m > n$：没有精确解，只求最小误差。
  Overdetermined $m > n$: no exact solution, minimise the error.
- 直线：A 的两列是 x 和全 1。
  Line: the columns of A are x and ones.
- A 是 $m \times n$，没有逆矩阵。
  A is $m \times n$ and has no inverse.

3.2 用线性代数推导

3.2 Derivation using linear algebra

向量记号 / Vector notation

- Y 是观测值组成的列向量；A 的每一列代表一个预测变量（一个基函数在所有数据点的取值）。

  Y is the column vector of observed values; each column of A represents a predictor variable (one basis function evaluated at all data points).

- 回归问题变成：在总平方误差最小的意义下"解" $A\beta = Y$。

  The regression problem becomes "solving" $A\beta = Y$ by minimising the total squared error.

关键方程：总平方误差是

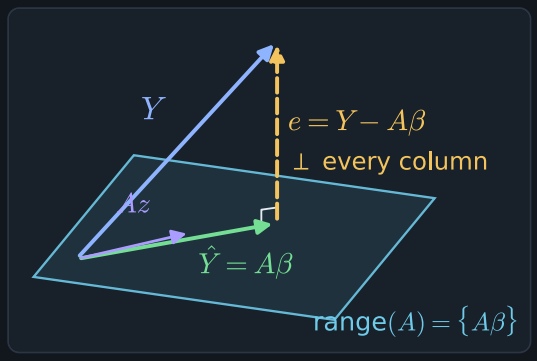
Key equation: the total squared error is

$$E = \|Y - A\beta\|^2$$

要让 $A\beta$ 尽量接近 Y。可以证明这导出（证明见 3.3）：

We seek the β that makes $A\beta$ as close as possible to Y. This leads to (proof on 3.3):

$$A^T(Y - A\beta) = 0$$



A 的列空间与 Y 的投影（示意）

/ range of A and the projection of Y (sketch)

解：展开得 $A^TY - A^TA\beta = 0$，即正规方程 $A^TA\beta = A^TY$。解出 β：

Solution: expanding gives $A^TY - A^TA\beta = 0$, the normal equation $A^TA\beta = A^TY$. Solving for β:

$$\beta = (A^TA)^{-1}A^TY$$

最小二乘回归公式；A 的列线性无关时 A^TA 可逆

/ the least squares formula; A^TA is invertible if A has linearly independent columns

几何意义 / Geometric meaning

不管 β 取什么，$A\beta$ 都落在 A 的列空间里（图中的平面）。平面上离 Y 最近的点是 Y 的正交投影 $\hat{Y}$，所以残差 $e = Y - A\beta$ 必须垂直于整个平面，也就是垂直于 A 的每一列 a_k。回想 $\mathrm{dot}(v, w) = v^Tw$：垂直就是 $a_k^Te = 0$。把 $k = 1, \ldots, n$ 这 n 个条件叠在一起，就是 $A^Te = 0$。

Whatever β is, $A\beta$ lies in the column space of A (the plane in the figure). The point of the plane closest to Y is its orthogonal projection $\hat{Y}$, so the residual $e = Y - A\beta$ must be perpendicular to the whole plane, i.e. to every column a_k of A. Recall $\mathrm{dot}(v, w) = v^Tw$: perpendicular means $a_k^Te = 0$. Stacking these n conditions for $k = 1, \ldots, n$ gives $A^Te = 0$.

考点速记 KEY POINTS

- 核心条件：$A^T(Y - A\beta) = 0$。
  Core condition: $A^T(Y - A\beta) = 0$.
- 残差垂直于 A 的每一列。
  The residual is orthogonal to every column of A.
- 正规方程 $A^TA\beta = A^TY$，A^TA 是 $n \times n$。
  Normal equation $A^TA\beta = A^TY$; A^TA is $n \times n$.
- $\mathrm{dot}(v, w) = v^Tw$。
  $\mathrm{dot}(v, w) = v^Tw$.

3.3 投影的证明与可逆条件 拓展 EXT

3.3 Proof of the projection and when A^TA is invertible

为什么 $A^Te=0$ 给出最小值 / Why $A^Te=0$ gives the minimum

设 β 满足 $A^Te=0$，$e=Y-A\beta$。任取另一组参数 z：

> Let β satisfy $A^Te=0$ with $e=Y-A\beta$. Take any other parameter vector z:

$$\begin{aligned} Y-Az &= (Y-A\beta)+A(\beta-z) = e+A(\beta-z) \\ \|Y-Az\|^2 &= \|e\|^2 + 2\,(\beta-z)^TA^Te + \|A(\beta-z)\|^2 \\ &= \|e\|^2 + \|A(\beta-z)\|^2 \;\geq\; \|e\|^2 \end{aligned}$$

中间项因 $A^Te=0$ 消失（勾股定理） / the cross term vanishes because $A^Te=0$ (Pythagoras)

所以没有任何 z 的误差比 β 更小。等号只在 $A(\beta-z)=0$ 时成立；A 的列线性无关时这意味着 $z=\beta$，即最小值**唯一**。

> So no z has a smaller error than β. Equality needs $A(\beta-z)=0$; with independent columns that means $z=\beta$, so the minimiser is **unique**.

A^TA 什么时候可逆 / When is A^TA invertible?

$$A^TAv=0 \;\Rightarrow\; v^TA^TAv=\|Av\|^2=0 \;\Rightarrow\; Av=0 \;\Rightarrow\; v=0 \quad (\text{independent columns})$$

- A^TA 是 $n\times n$ 对称矩阵；它可逆 ⇔ $\operatorname{rank}A=n$ ⇔ 基函数在数据点上线性无关。
  > A^TA is a symmetric $n\times n$ matrix; it is invertible ⇔ $\operatorname{rank}A=n$ ⇔ the basis functions are independent on the data.
- 至少需要 $m\geq n$ 个数据点。拟合直线时若所有 x_i 相同，x 列与全 1 列成比例，A^TA 奇异。
  > At least $m\geq n$ points are needed. Fitting a line to points that all share one x_i makes the x column proportional to the ones column, so A^TA is singular.

伪逆与投影矩阵 / Pseudoinverse and projection matrix

伪逆（教材 16.4） / pseudoinverse (textbook 16.4)

$$A^+=(A^TA)^{-1}A^T, \quad \beta=A^+Y$$

对应 `np.linalg.pinv(A)` / this is `np.linalg.pinv(A)`

投影（帽子）矩阵 / projection (hat) matrix

$$\hat{Y}=PY, \quad P=A(A^TA)^{-1}A^T, \quad P^2=P$$

投影两次等于投影一次 / projecting twice = projecting once

考点速记 KEY POINTS

- $\|Y-Az\|^2=\|e\|^2+\|A(\beta-z)\|^2$。
  $\|Y-Az\|^2=\|e\|^2+\|A(\beta-z)\|^2$.
- 伪逆 $A^+=(A^TA)^{-1}A^T$。
  Pseudoinverse $A^+=(A^TA)^{-1}A^T$.
- 列无关 ⇔ A^TA 可逆 ⇔ 解唯一。
  Independent columns ⇔ A^TA invertible ⇔ unique solution.
- 至少 $m\geq n$ 个点。
  Need at least $m\geq n$ points.

4.1 微积分方法：总误差

4.1 Calculus approach: the total error

回顾核心概念 / Recall the key concept

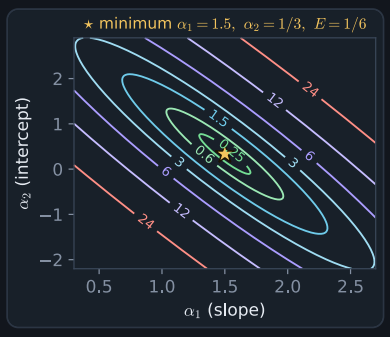
最小二乘回归的目标是找出一组参数 α_j，使总误差 E 最小；E 是预测值与实际数据点之差的平方和。

> The goal of least squares regression is to find the parameters α_j that minimise the total error E, the sum of squared differences between predicted values and actual data points.

总误差的公式 / Equation for total error

$$E = \sum_{i=1}^{m} \left(y_i - \sum_{j=1}^{n} \alpha_j f_j(x_i) \right)^2$$

符号 Symbol	含义 Meaning
m	数据点个数 · number of data points
n	基函数个数 · number of basis functions
α_j	未知参数 · unknown parameters
$f_j(x_i)$	基函数在 x_i 的值 · basis function at x_i
y_i	实际值 / 目标值 · actual / target values



4.3 例题的 $E(\alpha_1, \alpha_2)$ 等高线

/ contours of $E(\alpha_1, \alpha_2)$ for the example on 4.3

最小化的思路 / Concept of minimisation

对某一个 α_k 求 E 的偏导并令其为零；对 $k = 1, \ldots, n$ 都这样做，得到一个线性方程组，解它就让 E 最小。

> Set the partial derivative of E with respect to a specific α_k to zero; doing this for $k = 1, \ldots, n$ gives a system of linear equations whose solution minimises E.

为什么驻点就是最小值 拓展 EXT / Why the stationary point is the minimum

E 是 α 的二次函数（教材称为 n 维抛物面），二阶导数矩阵（Hessian）是 $2A^TA$，列无关时正定。所以只有一个驻点，它就是全局最小值；右图的等高线是一组同心椭圆。

> E is quadratic in α (the textbook calls it an n-dimensional paraboloid), with Hessian $2A^TA$, positive definite for independent columns. So there is exactly one stationary point and it is the global minimum; the contours in the figure are nested ellipses.

考点速记 KEY POINTS

- $E = \sum_i (y_i - \sum_j \alpha_j f_j(x_i))^2$。
  $E = \sum_i (y_i - \sum_j \alpha_j f_j(x_i))^2$.
- E 是二次函数，Hessian $= 2A^TA$。
  E is quadratic; Hessian $= 2A^TA$.
- m 个点（外层和），n 个参数（内层和）。
  m points (outer sum), n parameters (inner sum).
- 唯一驻点 = 全局最小。
  Unique stationary point = global minimum.

4.2 用多元微积分推导

4.2 Derivation using multivariable calculus

步骤 1：偏导数 / Step 1: partial derivatives

对每个参数 α_k，求误差函数 E 对 α_k 的偏导并令其为零（链式法则：内层和对 α_k 求导得 $f_k(x_i)$）：

For each parameter α_k, take the partial derivative of E with respect to α_k and set it to zero (chain rule: the inner sum differentiates to $f_k(x_i)$):

$$\frac{\partial E}{\partial \alpha_k} = -2\sum_{i=1}^{m}\Big(y_i - \sum_{j=1}^{n}\alpha_j f_j(x_i)\Big) f_k(x_i) = 0$$

整理（交换求和次序）：

Rearranging (swapping the order of the sums):

$$\sum_{j=1}^{n}\alpha_j \sum_{i=1}^{m} f_k(x_i) f_j(x_i) = \sum_{i=1}^{m} f_k(x_i)\, y_i, \qquad k = 1, \ldots, n$$

步骤 2：矩阵形式 / Step 2: matrix representation

对所有 k 写出这 n 个方程，方程组可以写成矩阵形式：

Considering all possible k, the system of equations can be written in matrix form:

$$A^T A\,\alpha = A^T Y$$

逐个元素 EXT：$(A^TA)_{kj} = \sum_i f_k(x_i) f_j(x_i)$，$(A^TY)_k = \sum_i f_k(x_i)\, y_i$，正好是上式的两边

/ entry by entry: $(A^TA)_{kj} = \sum_i f_k(x_i) f_j(x_i)$, $(A^TY)_k = \sum_i f_k(x_i)\, y_i$, exactly the two sides above

其中 A 是基函数在数据点上的取值矩阵，Y 是输出值列向量，α 是待求参数列向量。

Here A is the matrix of basis functions evaluated at the data points, Y the column vector of outputs, and α the column vector of parameters.

步骤 3：最终解 / Step 3: final solution

$$\alpha = (A^T A)^{-1} A^T Y$$

与 3.2 线性代数推导的结果完全相同 / identical to the linear-algebra result on 3.2

编者注 / Editorial note

幻灯片第 10 页把偏导写成 $+2\sum(y_i - \sum_j \alpha_j f_j(x_i)) f_k(x_i)$，少了负号。按第 9 页的定义 $E = \sum(y_i - \sum \alpha_j f_j)^2$，链式法则给出 $-2\sum(\cdots) f_k$。教材 §16.3 把 E 写成 $\sum(\hat{y}_i - y_i)^2$，所以那里是 $+2\sum(\sum_j \alpha_j f_j(x_i) - y_i) f_k(x_i)$。令偏导为零后两种写法得到同一组方程，结论不受影响。

Slide 10 writes the derivative as $+2\sum(y_i - \sum_j \alpha_j f_j(x_i)) f_k(x_i)$, missing a minus sign. With slide 9's $E = \sum(y_i - \sum \alpha_j f_j)^2$ the chain rule gives $-2\sum(\cdots) f_k$. Textbook §16.3 writes $E = \sum(\hat{y}_i - y_i)^2$, hence $+2\sum(\sum_j \alpha_j f_j(x_i) - y_i) f_k(x_i)$ there. Once set to zero both give the same equations, so the result is unaffected.

考点速记 KEY POINTS

- $\partial E/\partial \alpha_k = -2\sum_i e_i f_k(x_i) = 0$。
  $\partial E/\partial \alpha_k = -2\sum_i e_i f_k(x_i) = 0$.
- n 个方程、n 个未知数：$A^T A\alpha = A^T Y$。
  n equations, n unknowns: $A^T A\alpha = A^T Y$.
- 这正是 $A^T e = 0$ 的第 k 行。
  This is row k of $A^T e = 0$.
- 负号不影响解（令其为零）。
  The sign does not change the solution.

4.3 手算例题 拓展 EXT

4.3 Worked example by hand

用 $\hat{y} = \alpha_1 x + \alpha_2$ 拟合三个点 $(1,2),\ (2,3),\ (3,5)$。

> Fit $\hat{y} = \alpha_1 x + \alpha_2$ to the three points $(1,2),\ (2,3),\ (3,5)$.

组装 A、A^TA、A^TY / Assemble A, A^TA, A^TY

$$A = \begin{pmatrix} 1 & 1 \\ 2 & 1 \\ 3 & 1 \end{pmatrix}, \quad Y = \begin{pmatrix} 2 \\ 3 \\ 5 \end{pmatrix}, \quad A^TA = \begin{pmatrix} \sum x_i^2 & \sum x_i \\ \sum x_i & m \end{pmatrix} = \begin{pmatrix} 14 & 6 \\ 6 & 3 \end{pmatrix}, \quad A^TY = \begin{pmatrix} \sum x_i y_i \\ \sum y_i \end{pmatrix} = \begin{pmatrix} 23 \\ 10 \end{pmatrix}$$

解 2×2 方程组 / Solve the 2×2 system

$$\begin{pmatrix} a & b \\ c & d \end{pmatrix}^{-1} = \frac{1}{ad - bc} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}, \qquad \det(A^TA) = 14 \cdot 3 - 6 \cdot 6 = 6$$

$$\alpha_1 = \frac{3 \cdot 23 - 6 \cdot 10}{6} = \frac{9}{6} = 1.5, \qquad \alpha_2 = \frac{14 \cdot 10 - 6 \cdot 23}{6} = \frac{2}{6} = \frac{1}{3}, \qquad \hat{y} = 1.5x + \tfrac{1}{3}$$

检查 / Checks

x_i	y_i	$\hat{y}_i$	$e_i = y_i - \hat{y}_i$	$x_i e_i$	e_i^2
1	2	$11/6$	$+1/6$	$+1/6$	$1/36$
2	3	$10/3$	$-1/3$	$-2/3$	$4/36$
3	5	$29/6$	$+1/6$	$+1/2$	$1/36$
和 · sum			0	0	$E = 1/6$

- $\sum e_i = 0$ 和 $\sum x_i e_i = 0$ 就是 $A^Te = 0$ 的两行：残差垂直于 A 的两列。
  > $\sum e_i = 0$ and $\sum x_i e_i = 0$ are the two rows of $A^Te = 0$: the residual is orthogonal to both columns of A.
- 均值点：$\bar{x} = 2,\ \bar{y} = 10/3$，而 $1.5 \cdot 2 + 1/3 = 10/3$，直线过 $(\bar{x}, \bar{y})$。
  > Mean point: $\bar{x} = 2,\ \bar{y} = 10/3$, and $1.5 \cdot 2 + 1/3 = 10/3$, so the line passes through $(\bar{x}, \bar{y})$.
- Python 复算：`np.linalg.lstsq(A, Y, rcond=None)[0]` 给出 `[1.5, 0.33333333]`，残差平方和 $0.1\overline{6}$。
  > Python check: `np.linalg.lstsq(A, Y, rcond=None)[0]` returns `[1.5, 0.33333333]` with residual sum of squares $0.1\overline{6}$.

考点速记 KEY POINTS

- 直线：A^TA 的元素是 $\sum x_i^2,\ \sum x_i,\ m$。
  Line: the entries of A^TA are $\sum x_i^2,\ \sum x_i,\ m$.
- 2×2 逆：对调对角、副对角变号、除以行列式。
  2×2 inverse: swap diagonal, negate off-diagonal, divide by det.
- 验算：$\sum e_i = 0$、$\sum x_i e_i = 0$。
  Check: $\sum e_i = 0,\ \sum x_i e_i = 0$.
- 本例 $\hat{y} = 1.5x + 1/3,\ E = 1/6$。
  Here $\hat{y} = 1.5x + 1/3,\ E = 1/6$.

5.1 Python 中的最小二乘方法：工作流

5.1 Python methods for least squares: the workflow

关键 Python 库 / Key Python libraries

- 有几个 Python 库可以做最小二乘回归：NumPy 和 SciPy。
  > Several Python libraries can do least squares regression: NumPy and SciPy.
- **NumPy**：提供 `numpy.linalg.lstsq()` 和 `numpy.linalg.pinv()` 等函数求解最小二乘问题。
  > **NumPy**: functions such as `numpy.linalg.lstsq()` and `numpy.linalg.pinv()` solve the least squares problem.
- **SciPy**：`optimize.curve_fit()` 做更高级的拟合，包括非线性回归。
  > **SciPy**: `optimize.curve_fit()` does more advanced fitting, including nonlinear regression.

示例流程（NumPy）：生成数据 → 组装 A → 最小二乘求解 → 画图 / Example workflow (NumPy): generate data → form A → solve → plot

模型 $\hat{y} = \alpha_1 x + \alpha_2$。第一步在 $[0,1]$ 上取 101 个点，加上随 x 变大的噪声 $x \cdot U(0,1)$：

> Model $\hat{y} = \alpha_1 x + \alpha_2$. Step 1 takes 101 points on $[0,1]$ and adds noise $x \cdot U(0,1)$ that grows with x:

```
# generate x and y
x = np.linspace(0, 1, 101)
y = 1 + x + x * np.random.random(len(x))
```

第二步组装 A。`np.vstack` 把 `x` 和全 1 两行叠成形状 $(2, 101)$，`.T` 转置成 $(101, 2)$：第一列 x 对应 α_1，第二列 1 对应 α_2。`y[:, np.newaxis]` 把形状 `(101,)` 变成列向量 $(101, 1)$。

> Step 2 assembles A. `np.vstack` stacks the rows `x` and ones into shape $(2, 101)$; `.T` transposes to $(101, 2)$: column 1 is x (for α_1), column 2 is 1 (for α_2). `y[:, np.newaxis]` turns shape `(101,)` into a column vector $(101, 1)$.

```
# assemble matrix A
A = np.vstack([x, np.ones(len(x))]).T

# turn y into a column vector
y = y[:, np.newaxis]
```

第三步直接套公式 $\beta = (A^T A)^{-1} A^T Y$（左边幻灯片写的就是它）。从里往外读：`np.dot(A.T,A)` 是 2×2 的 $A^T A$，求逆后乘 A^T 得 2×101 的伪逆，再乘 `y` 得 2×1 的参数：

> Step 3 applies $\beta = (A^T A)^{-1} A^T Y$ directly (the formula on the slide's left). Read inside out: `np.dot(A.T,A)` is the 2×2 matrix $A^T A$; its inverse times A^T is the 2×101 pseudoinverse; times `y` gives the 2×1 parameters:

```
# Direct least square regression
alpha = np.dot((np.dot(np.linalg.inv(np.dot(A.T,A)),A.T)),y)
print(alpha)
```

```
[[1.459573  ]
 [1.02952189]]
```

第四步画图：蓝点是数据，红线是 $\alpha_1 x + \alpha_2$。`alpha[0]` 是斜率 $\alpha_1 \approx 1.4596$，`alpha[1]` 是截距 $\alpha_2 \approx 1.0295$。

> Step 4 plots: blue dots are the data, the red line is $\alpha_1 x + \alpha_2$. `alpha[0]` is the slope $\alpha_1 \approx 1.4596$, `alpha[1]` the intercept $\alpha_2 \approx 1.0295$.

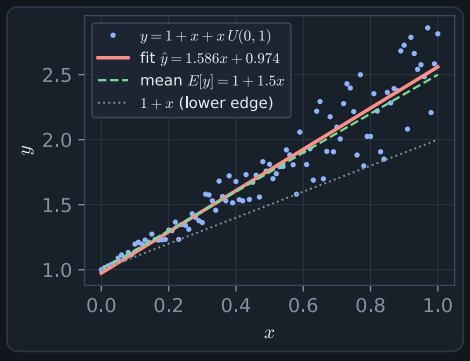
```
plt.plot(x, y, 'b.', x, alpha[0]*x + alpha[1], 'r')
```

考点速记 KEY POINTS

- `A` 形状 $(101, 2)$：列 x、列 1。
  `A` has shape $(101, 2)$: columns x and 1.
- `alpha[0]` 斜率，`alpha[1]` 截距（按列顺序）。
  `alpha[0]` slope, `alpha[1]` intercept (column order).
- `y[:, np.newaxis]`：`(101,)` → `(101,1)`。
  `y[:, np.newaxis]`: `(101,)` → `(101,1)`.
- 直接求逆只适合教学，实际用 `lstsq`（5.4）。
  Explicit inversion is for teaching; use `lstsq` (5.4).

5.2 结果解读：为什么斜率是 1.46 而不是 1

5.2 Reading the result: why the slope is 1.46, not 1



重画（固定种子 2026，一次随机抽样）

/ redrawn with seed 2026 (one random draw)

幻灯片得到 $\alpha_1 \approx 1.4596$、$\alpha_2 \approx 1.0295$。乍看斜率应为 1，因为代码写的是 `y = 1 + x + 噪声`。关键在于噪声 $x \cdot U(0,1)$ 的均值不是零：

> The slide gets $\alpha_1 \approx 1.4596$, $\alpha_2 \approx 1.0295$. At first sight the slope should be 1, since the code is `y = 1 + x + noise`. The point is that the noise $x \cdot U(0,1)$ does not have zero mean:

$$\mathbb{E}[y] = 1 + x + x \cdot \tfrac{1}{2} = 1 + 1.5x$$

所有数据都落在 $1 + x$ 与 $1 + 2x$ 之间，最小二乘拟合的是平均值那条线 $1 + 1.5x$。

> All data lie between $1 + x$ and $1 + 2x$; least squares recovers the mean line $1 + 1.5x$.

随机性有多大 拓展 EXT / How much does it vary?

情形 Case	斜率 α_1	截距 α_2
幻灯片（一次抽样）· slide, one draw	1.4596	1.0295
本讲义图（种子 2026）· figure above, seed 2026	1.5859	0.9739
2000 次重复的均值 ± 标准差 · mean ± std of 2000 draws	1.499 ± 0.062	1.001 ± 0.021
理论值 · theory	1.5	1

幻灯片的 1.4596 离 1.5 不到一个标准差，完全正常。代码没有设随机种子，所以每次运行结果都不一样（我照原样运行一次得到 1.4608、1.0168）。要复现结果，先固定随机数生成器：

> The slide's 1.4596 is within one standard deviation of 1.5, entirely normal. The code sets no random seed, so every run differs (running it unchanged once gave 1.4608, 1.0168). To make a result reproducible, fix the generator first:

```
rng = np.random.default_rng(0)        # reproducible noise
y = 1 + x + x * rng.random(len(x))
```

噪声越往右越大 拓展 EXT / Noise grows to the right

噪声的标准差是 $x/\sqrt{12}$，随 x 增大（异方差）。普通最小二乘对每个点一视同仁，结果仍然没有系统偏差，但不是最精确的估计。可以给每个点加权：`np.polyfit(x, y, 1, w=1/sigma)` 或 `curve_fit(..., sigma=sigma)`。

> The noise standard deviation is $x/\sqrt{12}$, growing with x (heteroscedastic). Ordinary least squares weighs every point equally; it stays unbiased but is not the most precise estimate. Points can be weighted: `np.polyfit(x, y, 1, w=1/sigma)` or `curve_fit(..., sigma=sigma)`.

考点速记 KEY POINTS

- 噪声 $x\,U(0,1)$ 均值 $x/2$ ⇒ 斜率 1.5。
  Noise $x\,U(0,1)$ has mean $x/2$ ⇒ slope 1.5.
- 无种子 ⇒ 每次结果不同。
  No seed ⇒ different result every run.
- 最小二乘拟合的是条件均值 $\mathbb{E}[y \mid x]$。
  Least squares fits the conditional mean $\mathbb{E}[y \mid x]$.
- 斜率抽样标准差约 0.06。
  Slope spread is about 0.06.

5.3 用 numpy.linalg.lstsq 和 scipy 的 curve_fit

5.3 Using numpy.linalg.lstsq and scipy's curve_fit

numpy.linalg.lstsq / numpy.linalg.lstsq

```
alpha = np.linalg.lstsq(A, y, rcond=None)[0]
print(alpha)
```

```
[[1.459573  ]
 [1.02952189]]
```

- `lstsq` 返回 4 个东西：解、残差平方和、A 的秩、A 的奇异值。`[0]` 只取解。
  > `lstsq` returns four things: the solution, the residual sum of squares, the rank of A and its singular values. `[0]` keeps only the solution.
- `rcond=None`：小于"机器精度 × max(m,n) × 最大奇异值"的奇异值当作零。NumPy 2.0 起这就是默认值；旧版本不写会给警告。
  > `rcond=None`: singular values below machine precision × max(m,n) × the largest one count as zero. This is the default since NumPy 2.0; older versions warned when it was omitted.
- 输出与 5.1 的直接求逆完全相同，因为用的是同一组数据。
  > The output matches the direct inverse on 5.1 exactly, because the data are the same.

```
coef, res, rank, sv = np.linalg.lstsq(A, y, rcond=None)
# coef: (2,1) solution | res: [sum of e_i^2] | rank: 2 | sv: singular values
```

optimize.curve_fit（不需要把 y 变成列向量） / optimize.curve_fit (no need to make y a column vector)

```
# generate x and y
x = np.linspace(0, 1, 101)
y = 1 + x + x * np.random.random(len(x))

def func(x, a, b):
    y = a*x + b
    return y

alpha = optimize.curve_fit(func, xdata = x, ydata = y)[0]
print(alpha)
```

```
[1.44331612 1.0396133 ]
```

- 模型函数的第一个参数必须是自变量 `x`，后面依次是待拟合参数 `a, b`。
  > The model function takes the independent variable `x` first, then the parameters `a, b`.
- `curve_fit` 返回 `(popt, pcov)`：最优参数和它们的协方差矩阵。`[0]` 取 `popt`，形状 `(2,)`。
  > `curve_fit` returns `(popt, pcov)`: optimal parameters and their covariance matrix. `[0]` keeps `popt`, shape `(2,)`.

为什么 curve_fit 得到 1.4433，而不是 1.4596 / Why curve_fit gives 1.4433, not 1.4596

这段代码**重新生成**了随机数据，差别来自数据，不是方法。在同一组数据上，`inv`、`pinv`、`lstsq`、`curve_fit` 四种方法给出相同结果（我用种子 2026 复算：全部是 1.585885、0.973912）。教材 16.4 的 `pinv` 写法：

> This cell **regenerates** the random data, so the difference comes from the data, not the method. On one data set, `inv`, `pinv`, `lstsq` and `curve_fit` agree (recomputed with seed 2026: all give 1.585885, 0.973912). The `pinv` version from textbook 16.4:

```
# textbook 16.4: the pseudoinverse (A^T A)^-1 A^T
pinv = np.linalg.pinv(A)
alpha = pinv.dot(y)
```

考点速记 KEY POINTS

- `lstsq(...)[0]`：返回元组的第 1 项是解。
  `lstsq(...)[0]`: item 1 of the tuple is the solution.
- `curve_fit(...)[0]` 是 `popt`，`(2,)`。
  `curve_fit(...)[0]` is `popt`, shape `(2,)`.
- 列向量 `y` ⇒ 解 $(2,1)$；一维 `y` ⇒ `(2,)`。
  Column `y` ⇒ solution $(2,1)$; 1-D `y` ⇒ `(2,)`.
- 同一数据，四种方法结果一致。
  Same data: all four methods agree.

5.4 为什么用 lstsq 而不直接求逆 拓展 EXT

5.4 Why lstsq rather than an explicit inverse

公式 $\beta = (A^TA)^{-1}A^TY$ 在纸上完全正确，但在计算机上直接照算会丢精度。原因是条件数（condition number）κ：它衡量输入的相对误差被放大多少倍。对 2-范数有

> The formula $\beta = (A^TA)^{-1}A^TY$ is exact on paper, but evaluating it literally on a computer loses accuracy. The reason is the condition number κ, which measures how much relative input error is amplified. In the 2-norm,

$$\kappa(A^TA) = \kappa(A)^2$$

双精度约 16 位有效数字；大约损失 $\log_{10}\kappa$ 位 / double precision has about 16 digits; roughly $\log_{10}\kappa$ of them are lost

数值实验：$[0,1]$ 上 101 个点的多项式基 $1, x, \dots, x^d$

/ Experiment: polynomial basis $1, x, \dots, x^d$ on 101 points in $[0,1]$

阶数 degree d	$\kappa(A)$	$\kappa(A^TA)$	求逆后剩下的有效位数 digits left via inverse
3	1.2×10^2	1.5×10^4	≈ 12
6	2.1×10^4	4.3×10^8	≈ 7
10	2.1×10^7	4.5×10^{14}	≈ 1

$d = 10$、真实系数全为 1、数据没有噪声时：`lstsq` 求出的系数最大误差 7.6×10^{-10}；`inv(A.T@A)@A.T@y` 的最大误差 0.62，几乎完全错了。

> With $d = 10$, true coefficients all 1 and noise-free data: `lstsq` recovers the coefficients with maximum error 7.6×10^{-10}; `inv(A.T@A)@A.T@y` has maximum error 0.62, essentially wrong.

实际做法 / What to do in practice

- 用 `np.linalg.lstsq`：它基于 A 的奇异值分解（SVD），直接处理 A，不形成 A^TA，精度由 $\kappa(A)$ 决定。
  > Use `np.linalg.lstsq`: it works on A through its singular value decomposition (SVD) without forming A^TA, so accuracy depends on $\kappa(A)$.
- 列线性相关时 `inv` 会报错或给出垃圾结果；`lstsq` 和 `pinv` 返回范数最小的那个解。
  > With dependent columns `inv` fails or returns garbage; `lstsq` and `pinv` return the minimum-norm solution.
- 高阶多项式：先把 x 中心化、缩放，或用 NumPy 文档推荐的 `np.polynomial.Polynomial.fit`（数值上更稳定）。
  > High-degree polynomials: centre and scale x first, or use `np.polynomial.Polynomial.fit`, which the NumPy docs recommend as numerically more stable.

考点速记 KEY POINTS

- $\kappa(A^TA) = \kappa(A)^2$。
  $\kappa(A^TA) = \kappa(A)^2$.
- 约损失 $\log_{10}\kappa$ 位有效数字。
  About $\log_{10}\kappa$ digits are lost.
- 不要显式求逆，用 `lstsq`（SVD）。
  Do not invert explicitly; use `lstsq` (SVD).
- 高阶多项式先中心化、缩放 x。
  Centre and scale x for high degrees.

6.1 非线性函数的最小二乘回归

6.1 Least squares regression for nonlinear functions

- **定义**：最小二乘回归通过让观测数据与预测数据之差的平方和最小来估计参数。

  > **Definition**: least squares regression estimates parameters by minimising the sum of squared differences between observed and predicted data points.

- **非线性函数**：有些函数不能用基函数的传统线性组合来表示，但仍然可以用最小二乘。

  > **Nonlinear functions**: some functions cannot be modelled as traditional linear combinations of basis functions but can still benefit from least squares regression.

非线性函数的两种方法 / Methods for nonlinear functions

1. **对数变换**：对非线性函数（如指数函数和幂函数）取对数，转换成线性形式（6.2）。

   > **Logarithmic transformations**: convert nonlinear functions (e.g. exponential and power functions) to a linear form by taking logarithms (6.2).

2. **直接非线性拟合**：用 scipy 的 `curve_fit` 等方法直接拟合非线性模型，不做变换（6.6）。

   > **Direct nonlinear fitting**: use methods like `curve_fit` from scipy to fit a nonlinear model without transforming it (6.6).

怎么判断用哪种方法 拓展 EXT / Which method for which model

模型 Model	对参数线性? Linear in parameters?	怎么拟合 How to fit
$a + bx + cx^2$	是 Yes	`lstsq` / `polyfit`，列 $1, x, x^2$
$a + b\sin x + c\,e^x$	是 Yes	`lstsq`，列 $1, \sin x, e^x$
$\alpha e^{\beta x}$	否 No	$\ln y = \ln\alpha + \beta x$，或 `curve_fit`
bx^m	否 No	$\ln y = m\ln x + \ln b$，或 `curve_fit`
$a\sin(bx)$	否 No	只能 `curve_fit`，需要好的初值 · only `curve_fit`, needs a good initial guess

判断技巧 / A quick test

对每个参数求偏导。若所有偏导都不含任何参数（例如 $\partial(a + b\sin x)/\partial b = \sin x$），模型对参数线性，可以一步解出。若偏导里还有参数（$\partial(\alpha e^{\beta x})/\partial\beta = \alpha x e^{\beta x}$），就是非线性的，要线性化或迭代。

> Differentiate with respect to each parameter. If no derivative contains a parameter (e.g. $\partial(a + b\sin x)/\partial b = \sin x$), the model is linear in its parameters and solves in one step. If a parameter remains ($\partial(\alpha e^{\beta x})/\partial\beta = \alpha x e^{\beta x}$), it is nonlinear: linearise or iterate.

考点速记 KEY POINTS

- 两条路：取对数线性化，或 `curve_fit`。
  Two routes: log-linearise, or `curve_fit`.
- $a + b\sin x$ 是线性的；$a\sin(bx)$ 不是。
  $a + b\sin x$ is linear; $a\sin(bx)$ is not.
- 偏导不含参数 ⇒ 线性模型。
  Derivatives free of parameters ⇒ linear model.
- 指数、幂函数可以取对数线性化。
  Exponential and power laws can be log-linearised.

6.2 非线性模型的对数技巧

6.2 Logarithmic tricks for nonlinear models

1. 指数函数 / 1. Exponential functions

给定指数函数，可以用对数把它线性化：

Given an exponential function, we can linearise it using logarithms:

原模型 / model

$$y = \alpha e^{\beta x}$$

取对数 / take logs

$$\log(y) = \log(\alpha) + \beta x$$

对线性化后的形式做最小二乘，再恢复参数。具体地，令 $\tilde{y} = \ln y$、$\tilde{\alpha} = \ln \alpha$，拟合直线 $\tilde{y} = \tilde{\alpha} + \beta x$（$A$ 的列是 x 和 1），得到斜率 β 和截距 $\tilde{\alpha}$，最后 $\alpha = e^{\tilde{\alpha}}$。

Perform least squares on the linearised form, then recover the parameters. Concretely, set $\tilde{y} = \ln y$ and $\tilde{\alpha} = \ln \alpha$, fit the line $\tilde{y} = \tilde{\alpha} + \beta x$ (columns of A: x and 1) to get slope β and intercept $\tilde{\alpha}$, then $\alpha = e^{\tilde{\alpha}}$.

```
# textbook 16.5: log trick on the same data
A = np.vstack([x, np.ones(len(x))]).T
beta, log_alpha = np.linalg.lstsq(A, np.log(y), rcond = None)[0]
alpha = np.exp(log_alpha)
print(f'alpha={alpha}, beta={beta}')
```

```
alpha=0.13233125271210974, beta=0.27092999595397904
```

2. 幂函数 / 2. Power functions

幂函数的做法类似：

A similar approach works for power functions:

原模型 / model

$$y = bx^m$$

取对数 / take logs

$$\log(y) = m\log(x) + \log(b)$$

变换后用最小二乘求参数 m 和 b：对 $(\ln x_i, \ln y_i)$ 拟合直线，斜率是 m，截距是 $\ln b$。例：$y = 3x^{1.5}$ 在 $x = 1, 2, 4, 8$ 上，`np.polyfit(np.log(x), np.log(y), 1)` 给出 $m = 1.5$、$b = e^{\text{截距}} = 3$。

After transforming, least squares gives m and b: fit a line to $(\ln x_i, \ln y_i)$; the slope is m and the intercept is $\ln b$. Example: $y = 3x^{1.5}$ at $x = 1, 2, 4, 8$; `np.polyfit(np.log(x), np.log(y), 1)` gives $m = 1.5$ and $b = e^{\text{intercept}} = 3$.

注意 / Caution

取对数要求 $y > 0$（幂函数还要求 $x > 0$）；数据里有零或负值时 `np.log` 给出 `-inf` 或 `nan`。`np.log` 是自然对数；用 `log10` 也可以，但恢复时要用 $10^{\text{截距}}$。

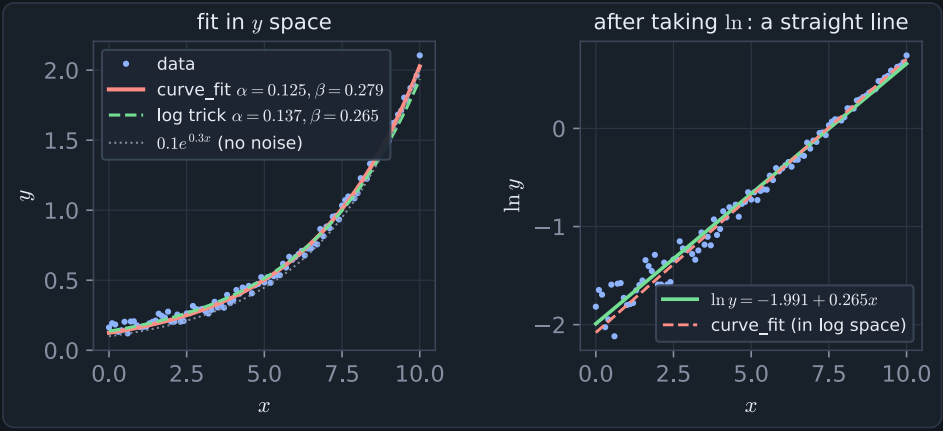
Taking logs needs $y > 0$ (and $x > 0$ for a power law); zeros or negatives in the data make `np.log` return `-inf` or `nan`. `np.log` is the natural log; `log10` also works, but then recover with $10^{\text{intercept}}$.

考点速记 KEY POINTS

- $y = \alpha e^{\beta x}$：对 $(x, \ln y)$ 拟直线，$\alpha = e^{\text{截距}}$。
  $y = \alpha e^{\beta x}$: line through $(x, \ln y)$, $\alpha = e^{\text{intercept}}$.
- 要求 $y > 0$（幂函数还要 $x > 0$）。
  Needs $y > 0$ (and $x > 0$ for powers).
- $y = bx^m$：对 $(\ln x, \ln y)$ 拟直线，斜率 m。
  $y = bx^m$: line through $(\ln x, \ln y)$, slope m.
- 别忘了把截距指数化还原。
  Remember to exponentiate the intercept back.

6.3 对数技巧和直接拟合为什么结果不同 拓展 EXT

6.3 Why the log trick and the direct fit disagree



同一组数据：左为原坐标，右为取对数后（固定种子 7 重画） / one data set: original axes (left) and after logs (right), redrawn with seed 7

方法 Method	α	β	最小化的量 Minimises
生成数据的真值 · true values	0.1	0.3	—
对数技巧（教材 16.5）· log trick	0.1323	0.2709	$\sum(\ln y_i - \ln \hat{y}_i)^2$
`curve_fit`（幻灯片 6）· direct fit	0.1242	0.2801	$\sum(y_i - \hat{y}_i)^2$

原因 1：最小化的不是同一个量 / Reason 1: they minimise different things

$\ln y_i - \ln \hat{y}_i \approx (y_i - \hat{y}_i)/y_i$，所以对数技巧近似最小化**相对**误差，小 y 的点权重变大；`curve_fit` 最小化原坐标里的绝对误差，由大 y 的点主导。两者的"最佳"本来就不一样。

> $\ln y_i - \ln \hat{y}_i \approx (y_i - \hat{y}_i)/y_i$, so the log trick roughly minimises **relative** error and gives small-y points more weight; `curve_fit` minimises absolute error in the original axes, dominated by large-y points. Their "best" fits differ by design.

原因 2：噪声的均值不是零 / Reason 2: the noise does not have zero mean

数据是 $y = 0.1e^{0.3x} + 0.1\,U(0,1)$，噪声均值是 $+0.05$，而模型里没有常数项来吸收它，所以两种方法都偏离 $(0.1, 0.3)$。验证：对**无噪声**的 $0.1e^{0.3x} + 0.05$ 拟合，`curve_fit` 得 $(0.1244, 0.2794)$，对数技巧得 $(0.1373, 0.2650)$；500 次随机重复的平均值是 $(0.1245, 0.2793)$ 和 $(0.1362, 0.2660)$。偏差几乎全部来自这个 0.05 的偏移。在 x 小的地方 0.05 和 $0.1e^{0.3x}$ 同一个量级，对数技巧受的影响更大。

> The data are $y = 0.1e^{0.3x} + 0.1\,U(0,1)$; the noise mean is $+0.05$ and the model has no constant term to absorb it, so both methods miss $(0.1, 0.3)$. Check: fitting the **noise-free** $0.1e^{0.3x} + 0.05$, `curve_fit` gives $(0.1244, 0.2794)$ and the log trick $(0.1373, 0.2650)$; averages over 500 random draws are $(0.1245, 0.2793)$ and $(0.1362, 0.2660)$. Nearly all the bias comes from the 0.05 offset. At small x, 0.05 is as large as $0.1e^{0.3x}$, so the log trick suffers more.

考点速记 KEY POINTS

- 对数技巧 ≈ 最小化相对误差。
  Log trick ≈ minimises relative error.
- 噪声均值 0.05 ⇒ 两种方法都有偏差。
  Noise mean 0.05 ⇒ both are biased.
- `curve_fit` 最小化原坐标的平方误差。
  `curve_fit` minimises squared error in y.
- 对数结果适合作为 `curve_fit` 的初值。
  The log result is a good initial guess for `curve_fit`.

6.4 其他非线性回归技巧（1）：多项式回归

6.4 Other techniques for nonlinear regression (1): polynomial regression

用最小二乘拟合不同阶数的多项式：

> Use least squares for polynomials of varying orders:

$$f(x) = a_n x^n + a_{n-1}x^{n-1} + \cdots + a_2 x^2 + a_1 x^1 + a_0$$

对 x 非线性，对系数 a_k 线性：A 的列是 $x^n, \ldots, x, 1$（范德蒙德矩阵）

/ nonlinear in x, linear in the a_k: the columns of A are $x^n, \ldots, x, 1$ (a Vandermonde matrix)

阶数越高越灵活，但有**过拟合**（overfitting）的风险。

> Higher-order polynomials are more flexible but risk **overfitting**.

```
x_d = np.array([0, 1, 2, 3, 4, 5, 6, 7, 8])
y_d = np.array([0, 0.8, 0.9, 0.1, -0.6, -0.8, -1, -0.9, -0.4])

plt.figure(figsize = (12, 8))
for i in range(1, 7):

    # get the polynomial coefficients
    y_est = np.polyfit(x_d, y_d, i)
    plt.subplot(2,3,i)
    plt.plot(x_d, y_d, 'o')
    # evaluate the values for a polynomial
    plt.plot(x_d, np.polyval(y_est, x_d))
    plt.title(f'Polynomial order {i}')

plt.tight_layout()
plt.show()
```

逐行说明 / Line by line

- `np.polyfit(x_d, y_d, i)`：i 阶最小二乘多项式拟合，返回 $i+1$ 个系数，**最高次在前**：$[a_i, \ldots, a_1, a_0]$。例：1 阶返回 $[-0.19,\ 0.5489]$，即 $f(x) = -0.19x + 0.5489$。
  > `np.polyfit(x_d, y_d, i)`: degree-i least-squares fit; returns $i+1$ coefficients, **highest power first**: $[a_i, \ldots, a_1, a_0]$. Order 1 returns $[-0.19,\ 0.5489]$, i.e. $f(x) = -0.19x + 0.5489$.
- `np.polyval(y_est, x_d)`：用这些系数在 `x_d` 处求多项式的值（系数顺序与 `polyfit` 一致）。
  > `np.polyval(y_est, x_d)`: evaluates the polynomial with those coefficients at `x_d` (same coefficient order as `polyfit`).
- `plt.subplot(2,3,i)`：2 行 3 列的第 i 个子图，`range(1, 7)` 画 1 到 6 阶。
  > `plt.subplot(2,3,i)`: panel i of a 2×3 grid; `range(1, 7)` draws orders 1 to 6.
- 内部做的就是 A（列 $x^i, \ldots, 1$）的线性最小二乘，没有迭代。
  > Internally it is linear least squares on A (columns $x^i, \ldots, 1$), with no iteration.

考点速记 KEY POINTS

- 多项式回归 = 线性最小二乘（范德蒙德 A）。
  Polynomial regression = linear LS (Vandermonde A).
- `polyfit` 系数最高次在前。
  `polyfit` coefficients: highest power first.
- n 阶有 $n+1$ 个系数。
  Degree n has $n+1$ coefficients.
- 阶数高 ⇒ 灵活但易过拟合。
  Higher order ⇒ flexible but overfits.

6.5 阶数与过拟合 拓展 EXT

6.5 Polynomial order and overfitting



幻灯片的 9 个点；绿线在细网格上求值，黄虚线是幻灯片的画法

/ the slide's 9 points; green: evaluated on a fine grid, yellow dashed: the slide's way

阶数 order	SSE $= \sum e_i^2$
1	1.8629
2	1.8038
3	0.2595
4	0.0696
5	0.0654
6	0.0227
7	0.0021
8	**0（插值 interpolation）**

- 幻灯片只在 9 个数据点上求值再连线（`polyval(y_est, x_d)`），所以曲线画成了折线。要看真实形状，在细网格上求值：

  The slide evaluates only at the 9 data points and joins them (`polyval(y_est, x_d)`), so the curves appear as broken lines. To see the real shape, evaluate on a fine grid:

```
xf = np.linspace(0, 8, 400)            # fine grid
plt.plot(xf, np.polyval(y_est, xf))     # the real polynomial curve
```

- 3 阶已经抓住了形状：SSE 从 1.80 降到 0.26。

  Order 3 already captures the shape: SSE drops from 1.80 to 0.26.

- SSE 随阶数**单调不增**：高阶模型包含所有低阶模型（令高次系数为 0），最小值只会更小。所以不能靠训练数据的 SSE 选阶数。

  SSE **never increases** with order: a higher-order model contains every lower-order one (set the top coefficients to 0), so its minimum can only be lower. The training SSE therefore cannot choose the order.

- 9 个点、8 阶多项式有 9 个系数，曲线穿过每个点（SSE = 0）：这是插值（C17），不是回归，噪声也被一起拟合了。

  With 9 points, a degree-8 polynomial has 9 coefficients and passes through every point (SSE = 0): that is interpolation (C17), not regression, and it fits the noise too.

- 选阶数的办法：留出一部分数据做验证（交叉验证），或由物理模型决定阶数。

  Choose the order with held-out data (cross-validation), or let the physical model decide.

考点速记 KEY POINTS

- SSE 随阶数单调不增，不能用来选阶。
  SSE never rises with order; it cannot pick the order.
- 画多项式要在细网格上求值。
  Plot polynomials on a fine grid.
- m 个点、$m-1$ 阶 ⇒ 插值，SSE = 0。
  m points, degree $m-1$ ⇒ interpolation, SSE = 0.
- 用验证数据或物理模型选阶数。
  Choose the order by validation or physics.

6.6 其他非线性回归技巧（2）：用 curve_fit 直接拟合

6.6 Other techniques (2): direct nonlinear fitting with curve_fit

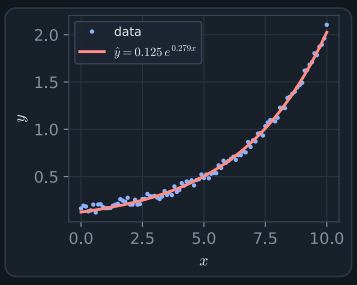
- scipy 的 `curve_fit` 直接估计非线性函数的参数，不需要做变换。
  > The `curve_fit` function from scipy directly estimates parameters of nonlinear functions without requiring transformations.
- 例：拟合指数函数 $y = \alpha e^{\beta x}$。幻灯片没有给出数据，数据来自教材 16.5：
  > Example: fitting an exponential function $y = \alpha e^{\beta x}$. The slide does not show the data; it comes from textbook 16.5:

```
# data behind slide 6 (textbook 16.5; not shown on the slide)
x = np.linspace(0, 10, 101)
y = 0.1*np.exp(0.3*x) + 0.1*np.random.random(len(x))
```

```
# let's define the function form
def func(x, a, b):
    y = a*np.exp(b*x)
    return y

alpha, beta = optimize.curve_fit(func, xdata = x, ydata = y)[0]
print(f'alpha={alpha}, beta={beta}')
```

```
alpha=0.12420824748558632, beta=0.28005793982974525
```



重画（种子 7：$\alpha = 0.1248,\ \beta = 0.2787$） / redrawn, seed 7

```
# Let's have a look of the data
plt.figure(figsize = (10,8))
plt.plot(x, y, 'b.')
plt.plot(x, alpha*np.exp(beta*x), 'r')
plt.xlabel('x')
plt.ylabel('y')
plt.show()
```

curve_fit 在做什么 拓展 EXT / What curve_fit does

- 它是**迭代**算法：从初值 `p0`（默认全为 1）出发，无边界约束时默认用 Levenberg–Marquardt（`method='lm'`），一步步减小 $\sum(y_i - f(x_i))^2$。
  > It is **iterative**: starting from `p0` (all ones by default), it uses Levenberg–Marquardt (`method='lm'`, the default without bounds) to reduce $\sum(y_i - f(x_i))^2$ step by step.
- `alpha, beta = ...[0]` 把 `popt` 的两个元素解包；第二个返回值 `pcov` 给出参数的不确定度。种子 7 的数据：$\alpha = 0.1248 \pm 0.0023$，$\beta = 0.2787 \pm 0.0022$。
  > `alpha, beta = ...[0]` unpacks the two entries of `popt`; the second return value `pcov` gives the uncertainties. Seed-7 data: $\alpha = 0.1248 \pm 0.0023$, $\beta = 0.2787 \pm 0.0022$.

```
popt, pcov = optimize.curve_fit(func, x, y, p0=[0.1, 0.3])
perr = np.sqrt(np.diag(pcov))     # 1-sigma errors of a, b
```

注意 / Caution

初值差时可能收敛到局部最小值，或迭代失败抛出 `RuntimeError`（例如 $a\sin(bx)$）。用对数技巧的结果作 `p0` 通常就够好。

> A poor initial guess can converge to a local minimum or fail with `RuntimeError` (e.g. for $a\sin(bx)$). The log-trick result usually makes a good enough `p0`.

考点速记 KEY POINTS

- `curve_fit` 迭代求解，默认 `p0` 全为 1。
  `curve_fit` iterates; default `p0` is all ones.
- 返回 `(popt, pcov)`；误差 $= \sqrt{\mathrm{diag}(\mathrm{pcov})}$。
  Returns `(popt, pcov)`; errors $= \sqrt{\mathrm{diag}(\mathrm{pcov})}$.
- 幻灯片结果 $\alpha = 0.1242,\ \beta = 0.2801$。
  Slide result $\alpha = 0.1242,\ \beta = 0.2801$.
- 好初值避免局部最小与失败。
  A good guess avoids local minima and failure.

6.7 方法对比与选择 拓展 EXT

6.7 Comparing and choosing the methods

方法 Method	适用模型 Models	最小化 Minimises	迭代 Iterative	注意 Watch out
`inv(A.T@A)@A.T@y`	对参数线性 · linear in parameters	$\sum e_i^2$	否 No	条件数平方，不推荐 · squares κ; avoid
`lstsq` / `pinv`	对参数线性 · linear in parameters	$\sum e_i^2$	否 No	SVD，稳定，可处理秩亏 · stable, handles rank deficiency
`polyfit`	多项式 · polynomials	$\sum e_i^2$	否 No	最高次在前；高阶病态 · highest power first; ill-conditioned at high degree
对数技巧 + `lstsq` · log trick	$\alpha e^{\beta x}$，bx^m	$\sum(\ln y_i - \ln \hat{y}_i)^2$	否 No	需 $y > 0$；权重改变 · needs $y > 0$; reweights points
`curve_fit`	任意 · any	$\sum e_i^2$（或带 `sigma` 的 χ^2）	是 Yes	需要初值；可能不收敛；给出 `pcov` · needs `p0`; may fail; gives `pcov`

决策顺序 / Decision order

1. 模型对参数线性吗？是 → 组装 A，用 `lstsq`（多项式可用 `polyfit`）。一步得到唯一解。
   > Is the model linear in its parameters? Yes → build A and use `lstsq` (or `polyfit` for polynomials). One step, unique solution.
2. 能取对数变成线性吗（指数、幂函数，$y > 0$）？能 → 对数技巧，快速得到近似值。
   > Can logs make it linear (exponential, power law, $y > 0$)? Yes → log trick for a quick estimate.
3. 需要在原坐标里误差最小，或模型无法线性化 → `curve_fit`，用第 2 步的结果作 `p0`。
   > Need the smallest error in the original axes, or the model cannot be linearised → `curve_fit`, with step 2's result as `p0`.
4. 最后画残差图：残差应在 0 附近随机分布；有明显规律说明模型选错了。
   > Finally plot the residuals: they should scatter randomly around 0; a visible pattern means the wrong model.

考试提醒 / Exam reminder

常见问法："哪个模型能直接用线性最小二乘？"看参数是否以线性方式出现，与 x 的形式无关。"为什么两种方法结果不同？"答：数据不同（重新生成随机数），或最小化的量不同（对数空间 vs 原空间）。

> Typical questions: "Which model can be fitted by linear least squares directly?" Check whether the parameters enter linearly; the form in x does not matter. "Why do two methods give different numbers?" Different data (regenerated random numbers), or a different objective (log space vs original space).

考点速记 KEY POINTS

- 线性 → `lstsq`；可取对数 → 对数技巧；否则 `curve_fit`。
  Linear → `lstsq`; log-able → log trick; else `curve_fit`.
- 只有 `curve_fit` 需要初值和迭代。
  Only `curve_fit` needs a guess and iteration.
- 对数技巧改变了各点的权重。
  The log trick changes the point weights.
- 拟合完一定看残差图。
  Always check the residual plot.

公式总表
Formula sheet

模型与误差 / model and error

$$\hat{y} = \sum_{j=1}^{n} \alpha_j f_j(x), \quad E = \|Y - A\beta\|_2^2$$

设计矩阵 / design matrix

$$A_{ij} = f_j(x_i), \quad A \in \mathbb{R}^{m\times n},\ m \geq n$$

正规方程 / normal equation

$$A^T A\beta = A^T Y, \quad \beta = (A^T A)^{-1} A^T Y$$

要求 A 列线性无关 / needs independent columns

正交条件 / orthogonality

$$A^T e = 0, \quad e = Y - A\beta$$

有截距 $\Rightarrow \sum e_i = 0$ / intercept $\Rightarrow \sum e_i = 0$

微积分形式 / calculus form

$$\frac{\partial E}{\partial \alpha_k} = -2\sum_i e_i\, f_k(x_i) = 0$$

过原点 $y = kx$ / through origin

$$k = \frac{\sum x_i y_i}{\sum x_i^2}$$

直线 $y = mx + b$（N 个点） / line (N points)

$$m = \frac{N\sum x_i y_i - \sum x_i \sum y_i}{N\sum x_i^2 - (\sum x_i)^2},\ b = \bar{y} - m\bar{x}$$

直线的 $A^T A$ / $A^T A$ for a line

$$A^T A = \begin{pmatrix} \sum x_i^2 & \sum x_i \\ \sum x_i & N \end{pmatrix},\ A^T Y = \begin{pmatrix} \sum x_i y_i \\ \sum y_i \end{pmatrix}$$

指数函数 / exponential

$$y = \alpha e^{\beta x} \Rightarrow \ln y = \ln\alpha + \beta x$$

$\alpha = e^{\text{截距}}$，需 $y > 0$ / $\alpha = e^{\text{intercept}}$, needs $y > 0$

幂函数 / power law

$$y = bx^m \Rightarrow \ln y = m\ln x + \ln b$$

对 $(\ln x, \ln y)$ 拟直线 / line through $(\ln x, \ln y)$

多项式 / polynomial

$$f(x) = a_n x^n + \cdots + a_1 x + a_0$$

A 列 $x^n, \ldots, 1$；m 点 $m-1$ 阶 = 插值

/ columns $x^n, \ldots, 1$; degree $m-1$ = interpolation

条件数 / conditioning

$$\kappa(A^T A) = \kappa(A)^2$$

用 `lstsq`，不要显式求逆

/ use `lstsq`, not an explicit inverse

Python 速查 / Python quick reference

调用 Call	返回 Returns	要点 Note
`np.vstack([x, np.ones(len(x))]).T`	A，形状 $(m, 2)$	列顺序决定参数顺序 · column order = parameter order
`np.linalg.lstsq(A, y, rcond=None)`	(解, 残差平方和, 秩, 奇异值) · (x, res, rank, s)	`[0]` 取解 · `[0]` is the solution
`np.linalg.pinv(A) @ y`	解 · solution	伪逆，SVD · pseudoinverse via SVD
`np.polyfit(x, y, n)` / `np.polyval(p, x)`	$n+1$ 个系数 · coefficients	最高次在前 · highest power first
`optimize.curve_fit(f, x, y, p0=...)`	`(popt, pcov)`	迭代；`p0` 默认全 1；误差 $\sqrt{\text{diag}(\text{pcov})}$

考点速记 KEY POINTS

- $A^T A\beta = A^T Y$，列无关时唯一解。
  $A^T A\beta = A^T Y$; unique for independent columns.
- $A^T e = 0$：残差垂直于每一列。
  $A^T e = 0$: residual $\perp$ every column.
- 指数/幂函数先取对数，记得还原截距。
  Exp/power: take logs, then undo the intercept.
- `polyfit` 最高次在前；`curve_fit` 要初值。
  `polyfit`: highest first; `curve_fit`: needs a guess.

01 / 6 过原点拟合 · Through-origin fit

弹簧刚度的手算

Spring stiffness by hand

弹簧被拉伸 $x = 1, 2, 3, 4$ cm，测得的力是 $F = 52, 98, 155, 197$ N。(a) 用最小二乘拟合 $F = kx$。(b) 求残差，验证 $\sum x_i e_i = 0$，并计算 $\sum e_i$。(c) 如果模型改成 $F = kx + b$，会怎样?

A spring is stretched by $x = 1, 2, 3, 4$ cm and the measured forces are $F = 52, 98, 155, 197$ N. (a) Fit $F = kx$ by least squares. (b) Find the residuals, check $\sum x_i e_i = 0$ and evaluate $\sum e_i$. (c) What changes with the model $F = kx + b$?

答案 Answer $k \approx 50.03$ N/cm

$k = 1501/30$；$\sum x_i e_i = 0$，但 $\sum e_i = 1.67 \neq 0$。

$k = 1501/30$; $\sum x_i e_i = 0$ **but** $\sum e_i = 1.67 \neq 0$.

(a) 求 k / (a) Find k

$$\sum x_i^2 = 1 + 4 + 9 + 16 = 30, \qquad \sum x_i F_i = 52 + 196 + 465 + 788 = 1501$$

$$k = \frac{1501}{30} = 50.033 \text{ N/cm}$$

(b) 残差 / (b) Residuals

x_i	F_i	kx_i	$e_i = F_i - kx_i$	$x_i e_i$
1	52	50.033	+1.967	+1.967
2	98	100.067	−2.067	−4.133
3	155	150.100	+4.900	+14.700
4	197	200.133	−3.133	−12.533
和 · sum			+1.667	0.000

$E = \sum e_i^2 = 41.97$ N^2。只有一个基函数 $f(x) = x$，所以正交条件只有一条：$\sum x_i e_i = 0$。没有截距列，$\sum e_i$ 不必为零。

$E = \sum e_i^2 = 41.97$ N^2. There is one basis function $f(x) = x$, hence one orthogonality condition, $\sum x_i e_i = 0$. With no intercept column, $\sum e_i$ need not vanish.

(c) 加截距 / (c) With an intercept

$\bar{x} = 2.5$, $\bar{F} = 125.5$, $\sum (x_i - \bar{x})^2 = 5$, $\sum (x_i - \bar{x})(F_i - \bar{F}) = 246$，所以 $k = 246/5 = 49.2$ N/cm，$b = 125.5 - 49.2 \cdot 2.5 = 2.5$ N。此时 $\sum e_i = 0$。物理上弹簧在 $x = 0$ 时 $F = 0$，$b = 2.5$ N 应视为测量误差，(a) 的模型更合理。

$\bar{x} = 2.5$, $\bar{F} = 125.5$, $\sum (x_i - \bar{x})^2 = 5$, $\sum (x_i - \bar{x})(F_i - \bar{F}) = 246$, so $k = 246/5 = 49.2$ N/cm and $b = 125.5 - 49.2 \cdot 2.5 = 2.5$ N. Now $\sum e_i = 0$. Physically $F = 0$ at $x = 0$, so $b = 2.5$ N is measurement error and model (a) is the better choice.

考点速记 KEY POINTS

- $k = \sum x_i F_i / \sum x_i^2 = 50.03$ N/cm。
  $k = \sum x_i F_i / \sum x_i^2 = 50.03$ N/cm.
- 加截距后 k 变成 49.2。
  With an intercept, k becomes 49.2.
- 无截距：$\sum x_i e_i = 0$，$\sum e_i \neq 0$。
  No intercept: $\sum x_i e_i = 0$, $\sum e_i \neq 0$.
- 选模型看物理：$F(0) = 0$。
  Pick the model from physics: $F(0) = 0$.

02 / 6 正规方程 · Normal equation

用正规方程手算一条直线

A line from the normal equation, by hand

用 $\hat{y} = \alpha_1 x + \alpha_2$ 拟合 $(0,1),(1,3),(2,4),(3,4)$。写出 A、A^TA、A^TY，解出参数，并给出残差和 E。

> **Fit $\hat{y} = \alpha_1 x + \alpha_2$ to $(0,1),(1,3),(2,4),(3,4)$. Write A, A^TA and A^TY, solve for the parameters, and give the residuals and E.**

答案 Answer $\hat{y} = x + 1.5,\ E = 1$

$A^TA = \begin{pmatrix} 14 & 6 \\ 6 & 4 \end{pmatrix}$，$A^TY = (23, 12)^T$，行列式 **20**。

> $A^TA = \begin{pmatrix} 14 & 6 \\ 6 & 4 \end{pmatrix}$, $A^TY = (23,12)^T$, **determinant 20.**

组装 / Assemble

$$A = \begin{pmatrix} 0 & 1 \\ 1 & 1 \\ 2 & 1 \\ 3 & 1 \end{pmatrix},\quad Y = \begin{pmatrix} 1 \\ 3 \\ 4 \\ 4 \end{pmatrix},\quad A^TA = \begin{pmatrix} \sum x_i^2 & \sum x_i \\ \sum x_i & 4 \end{pmatrix} = \begin{pmatrix} 14 & 6 \\ 6 & 4 \end{pmatrix},\quad A^TY = \begin{pmatrix} 23 \\ 12 \end{pmatrix}$$

求解 / Solve

$$\det = 14 \cdot 4 - 6 \cdot 6 = 20,\quad \alpha_1 = \frac{4 \cdot 23 - 6 \cdot 12}{20} = 1,\quad \alpha_2 = \frac{14 \cdot 12 - 6 \cdot 23}{20} = \frac{30}{20} = 1.5$$

用闭式解核对：$N = 4$，$\sum x = 6$，$\sum y = 12$，$\sum xy = 23$，$\sum x^2 = 14$，$m = (4 \cdot 23 - 6 \cdot 12)/(4 \cdot 14 - 36) = 20/20 = 1$，$b = \bar{y} - m\bar{x} = 3 - 1.5 = 1.5$。

> Check with the closed form: $N = 4$, $\sum x = 6$, $\sum y = 12$, $\sum xy = 23$, $\sum x^2 = 14$, $m = (4 \cdot 23 - 6 \cdot 12)/(4 \cdot 14 - 36) = 20/20 = 1$, $b = \bar{y} - m\bar{x} = 3 - 1.5 = 1.5$.

残差与检查 / Residuals and checks

x_i	y_i	$\hat{y}_i$	e_i	$x_i e_i$
0	1	1.5	−0.5	0
1	3	2.5	+0.5	+0.5
2	4	3.5	+0.5	+1.0
3	4	4.5	−0.5	−1.5
和 · sum			**0**	**0**

$E = 4 \times 0.25 = 1$。两个和都是零，即 $A^Te = 0$；Python 的 `np.linalg.lstsq` 给出同样的 $[1,\ 1.5]$。

> $E = 4 \times 0.25 = 1$. Both sums vanish, i.e. $A^Te = 0$; `np.linalg.lstsq` in Python returns the same $[1,\ 1.5]$.

考点速记 KEY POINTS

- A^TA 右下角是点数 N。
  The bottom-right entry of A^TA is N.
- 本题 $\hat{y} = x + 1.5$，$E = 1$。
  Here $\hat{y} = x + 1.5$, $E = 1$.
- 验算 $\sum e_i = 0$ 与 $\sum x_i e_i = 0$。
  Check $\sum e_i = 0$ and $\sum x_i e_i = 0$.
- 闭式解与矩阵解一致。
  Closed form and matrix solution agree.

03 / 6 线性与非线性 · Linear vs nonlinear

哪个模型能直接用线性最小二乘

Which model is linear least squares as it stands

未知参数是 a, b, c。下列哪个模型能直接用 `np.linalg.lstsq` 拟合（不做变换、不迭代）？

> **The unknown parameters are a, b, c. Which model can be fitted directly with `np.linalg.lstsq` (no transformation, no iteration)?**

A $y = a + bx^2 + c\sin x$ ✓ 正确 Correct

> $y = a + bx^2 + c\sin x$

B $y = a\,e^{bx}$

> $y = a\,e^{bx}$

C $y = a\,x^b$

> $y = a\,x^b$

D $y = a\sin(bx)$

> $y = a\sin(bx)$

答案 Answer A

A 对 a, b, c 线性：A 的三列是 1，x^2，$\sin x$。

> **A is linear in a, b, c: the columns of A are 1, x^2, $\sin x$.**

推导 / Derivation

判断的是参数，不是 x。A 的三个偏导 $\partial y/\partial a = 1$、$\partial y/\partial b = x^2$、$\partial y/\partial c = \sin x$ 都不含参数，所以 $y = A\beta$ 中 $\beta = (a, b, c)^T$，一步解出：

> The test concerns the parameters, not x. In A, the derivatives $\partial y/\partial a = 1$, $\partial y/\partial b = x^2$, $\partial y/\partial c = \sin x$ contain no parameters, so $y = A\beta$ with $\beta = (a, b, c)^T$ solves in one step:

$$A = \begin{pmatrix} 1 & x_1^2 & \sin x_1 \\ \vdots & \vdots & \vdots \\ 1 & x_m^2 & \sin x_m \end{pmatrix}, \qquad \beta = (A^T A)^{-1} A^T Y$$

干扰项为什么错 / Why the distractors are wrong

B b 在指数里，$\partial y/\partial b = axe^{bx}$ 含参数。可以先取对数 $\ln y = \ln a + bx$（需 $y > 0$），但那已经是变换，而且最小化的是对数误差；或用 `curve_fit`。

> b is in the exponent; $\partial y/\partial b = axe^{bx}$ contains parameters. Taking $\ln y = \ln a + bx$ (needs $y > 0$) is a transformation and minimises log error; otherwise use `curve_fit`.

C 幂函数：$\ln y = \ln a + b\ln x$（需 $x, y > 0$），同样要先变换。

> Power law: $\ln y = \ln a + b\ln x$ (needs $x, y > 0$), again a transformation first.

D b 在正弦里，取对数也无法线性化，只能用 `curve_fit` 迭代，且需要好的初值（误差函数有很多局部最小值）。

> b sits inside the sine and no log can linearise it; only `curve_fit` works, and it needs a good initial guess (the error has many local minima).

考点速记 KEY POINTS

- 线性 = 对参数线性，与 x 的形式无关。
  Linear = linear in parameters, whatever x does.
- 偏导含参数 ⇒ 非线性。
  Derivative contains a parameter ⇒ nonlinear.
- 指数、幂：可取对数线性化。
  Exponential, power: log-linearisable.
- $a\sin(bx)$：只能迭代。
  $a\sin(bx)$: iteration only.

04 / 6 对数技巧 · Log trick

用对数技巧拟合指数函数

Fitting an exponential with the log trick

数据：$x = 0, 1, 2, 3$，$y = 1.0,\ 2.7,\ 7.4,\ 20.1$。用对数技巧拟合 $y = \alpha e^{\beta x}$。你实际最小化的是什么量？

> **Data:** $x = 0, 1, 2, 3$, $y = 1.0,\ 2.7,\ 7.4,\ 20.1$. **Fit $y = \alpha e^{\beta x}$ with the log trick. Which quantity did you actually minimise?**

答案 Answer $\hat{y} \approx 0.997\, e^{1.001x}$

最小化的是 $\sum(\ln y_i - \ln \hat{y}_i)^2$，不是 $\sum(y_i - \hat{y}_i)^2$。

> **It minimises $\sum(\ln y_i - \ln \hat{y}_i)^2$, not $\sum(y_i - \hat{y}_i)^2$.**

步骤 1：取对数 / Step 1: take logs

$$\tilde{y}_i = \ln y_i = 0,\ 0.9933,\ 2.0015,\ 3.0007 \qquad (\text{model } \tilde{y} = \tilde{\alpha} + \beta x)$$

步骤 2：对 $(x, \tilde{y})$ 拟直线 / Step 2: fit a line to $(x, \tilde{y})$

$$A^T A = \begin{pmatrix} 14 & 6 \\ 6 & 4 \end{pmatrix}, \quad A^T \tilde{Y} = \begin{pmatrix} \sum x_i \tilde{y}_i \\ \sum \tilde{y}_i \end{pmatrix} = \begin{pmatrix} 13.9984 \\ 5.9955 \end{pmatrix}$$

$$\beta = \frac{4(13.9984) - 6(5.9955)}{20} = 1.0010, \qquad \tilde{\alpha} = \frac{14(5.9955) - 6(13.9984)}{20} = -0.0027$$

步骤 3：还原 / Step 3: transform back

$$\alpha = e^{\tilde{\alpha}} = e^{-0.0027} = 0.9973, \qquad \hat{y} = 0.9973\, e^{1.0010x}$$

数据其实接近 e^x，结果合理。直接用 `curve_fit` 拟合得 $\alpha = 0.9983$、$\beta = 1.0008$，在原坐标的平方误差是 3.8×10^{-4}，比对数技巧的 4.7×10^{-4} 小：这正是 `curve_fit` 最小化的量。

> The data are close to e^x, so the result makes sense. A direct `curve_fit` gives $\alpha = 0.9983$, $\beta = 1.0008$, with squared error 3.8×10^{-4} in the original axes, below the log trick's 4.7×10^{-4}: that is exactly what `curve_fit` minimises.

考点速记 KEY POINTS

- $\ln y = \ln \alpha + \beta x$：斜率 β，截距 $\ln \alpha$。
  $\ln y = \ln \alpha + \beta x$: slope β, intercept $\ln \alpha$.
- 对数技巧最小化对数空间的误差。
  The log trick minimises error in log space.
- 最后一步 $\alpha = e^{\text{截距}}$ 常被忘记。
  The last step $\alpha = e^{\text{intercept}}$ is often forgotten.
- 原坐标误差：`curve_fit` 更小。
  In original axes, `curve_fit` has the smaller error.

05 / 6 读代码 · Reading code

幻灯片代码中的形状与返回值

Shapes and return values in the slide code

设 `x = np.linspace(0, 1, 101)`，`A = np.vstack([x, np.ones(len(x))]).T`，`y = y[:, np.newaxis]`。下列哪个说法是**错误**的?

> **Let `x = np.linspace(0, 1, 101)`, `A = np.vstack([x, np.ones(len(x))]).T` and `y = y[:, np.newaxis]`. Which statement is false?**

A `A.shape` 是 `(101, 2)`。

> `A.shape` is `(101, 2)`.

B `alpha[0]` 是斜率，因为 `A` 的第一列是 `x`。

> `alpha[0]` is the slope, because the first column of `A` is `x`.

C `np.linalg.lstsq(A, y, rcond=None)` 只返回系数数组，所以 `[0]` 取出的是斜率。 ✓ 正确 Correct

> `np.linalg.lstsq(A, y, rcond=None)` returns only the coefficient array, so `[0]` picks out the slope.

D `optimize.curve_fit(func, x, y)[0]` 是形状 `(2,)` 的一维数组，`y` 可以保持一维。

> `optimize.curve_fit(func, x, y)[0]` is a 1-D array of shape `(2,)`, and `y` may stay 1-D.

答案 Answer C

`lstsq` **返回 4 元组；`[0]` 取的是整个解（形状 $(2,1)$），不是斜率。**

> `lstsq` **returns a 4-tuple; `[0]` takes the whole solution (shape $(2,1)$), not the slope.**

推导 / Derivation

`lstsq` 返回 `(x, residuals, rank, s)`：解、残差平方和、秩、奇异值。`[0]` 是元组的第一项，即解向量 `[[α₁],[α₂]]`；斜率要再取 `[0]`，即 `lstsq(...)[0][0]`。

> `lstsq` returns `(x, residuals, rank, s)`: solution, residual sum of squares, rank, singular values. `[0]` is the first tuple item, the solution vector `[[α₁],[α₂]]`; the slope needs another `[0]`, i.e. `lstsq(...)[0][0]`.

其他选项为什么对 / Why the other options are true

A `vstack` 得 $(2, 101)$，转置得 $(101, 2)$。

> `vstack` gives $(2, 101)$; the transpose is $(101, 2)$.

B 参数顺序跟列顺序一致。若写成 `np.vstack([np.ones(len(x)), x]).T`，`alpha[0]` 就变成截距。

> Parameter order follows column order. With `np.vstack([np.ones(len(x)), x]).T`, `alpha[0]` would be the intercept.

D `curve_fit` 返回 `(popt, pcov)`，`popt` 是一维的；幻灯片第 2 页也说明不需要把 `y` 变成列向量。

> `curve_fit` returns `(popt, pcov)` with a 1-D `popt`; slide 2 also says `y` need not be a column vector.

考点速记 KEY POINTS

- `lstsq` → (解, 残差, 秩, 奇异值)。
  `lstsq` → (solution, residuals, rank, s).
- 列顺序 = 参数顺序。
  Column order = parameter order.
- `curve_fit` → (popt, pcov)。
  `curve_fit` → (popt, pcov).
- 列向量 `y` ⇒ 解是 $(2, 1)$。
  Column `y` ⇒ solution is $(2, 1)$.

06 / 6 过拟合 · Overfitting

多项式阶数与过拟合

Polynomial order and overfitting

对 C16_2 第 5 页的 9 个数据点（$x_d = 0, 1, \ldots, 8$），下列哪个说法正确?

For the 9 data points on C16_2 slide 5 ($x_d = 0, 1, \ldots, 8$), which statement is correct?

A 6 阶是最好的模型，因为它在图中 6 个拟合里 SSE 最小。

Order 6 is the best model because it has the smallest SSE of the six fits shown.

B 8 阶多项式穿过全部 9 个点（SSE = 0），这是插值，不是回归。 ✓ 正确 Correct

A degree-8 polynomial passes through all 9 points (SSE = 0); that is interpolation, not regression.

C 在同一组数据上，提高阶数可能让 SSE 变大。

Raising the order can increase the SSE on the same data.

D `np.polyfit(x_d, y_d, 3)` 返回 $[a_0, a_1, a_2, a_3]$，最低次在前。

`np.polyfit(x_d, y_d, 3)` returns $[a_0, a_1, a_2, a_3]$, lowest power first.

答案 Answer B

m 个点、$m-1$ 阶有 m 个系数，能精确穿过每个点。

m points and degree $m-1$ give m coefficients, enough to hit every point.

推导 / Derivation

8 阶时 A 是 9×9 的范德蒙德矩阵（x_i 互不相同时可逆），$A\beta = Y$ 有精确解，SSE $= 0$。复算的 SSE：1 阶 1.863、3 阶 0.260、6 阶 0.0227、7 阶 0.0021、8 阶 0。曲线把噪声也拟合进去了，这就是过拟合。

At degree 8, A is a 9×9 Vandermonde matrix (invertible for distinct x_i), so $A\beta = Y$ has an exact solution and SSE $= 0$. Recomputed SSE: order 1 1.863, order 3 0.260, order 6 0.0227, order 7 0.0021, order 8 0. The curve fits the noise too: that is overfitting.

干扰项为什么错 / Why the distractors are wrong

A 训练 SSE 随阶数单调不增，最高阶总是"最小"，所以它不能用来选模型；要用验证数据或物理模型。

Training SSE never rises with order, so the highest order always "wins"; it cannot select the model. Use held-out data or physics.

C 高阶模型包含低阶模型（高次系数取 0 即可），最小 SSE 不会变大。

A higher-order model contains the lower one (set the top coefficients to 0), so the minimum SSE cannot grow.

D `polyfit` 返回的系数**最高次在前**：$[a_3, a_2, a_1, a_0] = [0.0329, -0.3811, 0.8906, 0.1253]$。

`polyfit` returns coefficients **highest power first**: $[a_3, a_2, a_1, a_0] = [0.0329, -0.3811, 0.8906, 0.1253]$.

考点速记 KEY POINTS

- m 点 $m-1$ 阶 ⇒ 插值，SSE = 0。
  m points, degree $m-1$ ⇒ interpolation, SSE = 0.
- 训练 SSE 单调不增，不能选阶。
  Training SSE never rises; it cannot pick the order.
- `polyfit`：最高次在前。
  `polyfit`: highest power first.
- 本例 3 阶已抓住形状（SSE 0.26）。
  Here order 3 already captures the shape (SSE 0.26).

答案速查与来源

Answer key and sources

题号 Q	答案 Answer	题号 Q	答案 Answer
1	$k = 50.03\ \mathrm{N/cm}$	2	$\hat{y} = x + 1.5,\ E = 1$
3	**A**	4	$\hat{y} = 0.997\, e^{1.001x}$
5	**C（错误的那项 · the false one）**	6	**B**

幻灯片里需要注意的地方 / Points to watch in the slides

- C16_1 第 10 页：偏导少了负号，应为 $-2\sum(y_i - \sum_j \alpha_j f_j) f_k$；令其为零后结论不变（4.2）。
  > C16_1 slide 10: the derivative is missing a minus sign, it is $-2\sum(y_i - \sum_j \alpha_j f_j) f_k$; the result is unchanged once it is set to zero (4.2).
- C16_1 第 6 页：E 是 L2 范数的**平方**（2.3）。
  > C16_1 slide 6: E is the **squared** L2 norm (2.3).
- C16_2 第 1–2 页：斜率约 1.46 是因为噪声均值为 $x/2$；curve_fit 的数字不同是因为数据重新生成（5.2、5.3）。
  > C16_2 slides 1–2: the slope is about 1.46 because the noise mean is $x/2$; curve_fit differs because the data were regenerated (5.2, 5.3).
- C16_2 第 5 页：曲线只在数据点上求值，画成了折线（6.5）。第 6 页：数据代码不在幻灯片上，来自教材 16.5（6.6）。
  > C16_2 slide 5: curves are evaluated only at the data points, so they look like broken lines (6.5). Slide 6: the data code is not on the slide; it is from textbook 16.5 (6.6).

来源 / Sources

1. Q. Kong, T. Siauw, A. Bayen, Python Programming and Numerical Methods, Ch. 16 Least Squares Regression (16.1–16.5)
   https://pythonnumericalmethods.studentorg.berkeley.edu/notebooks/chapter16.00-Least-Squares-Regression.html
2. NumPy documentation: numpy.linalg.lstsq
   https://numpy.org/doc/stable/reference/generated/numpy.linalg.lstsq.html
3. NumPy documentation: numpy.polyfit
   https://numpy.org/doc/stable/reference/generated/numpy.polyfit.html
4. SciPy documentation: scipy.optimize.curve_fit
   https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.curve_fit.html

图均按幻灯片的例子用 Matplotlib 3.11 重新绘制；数值由 Python（NumPy 2.5.4、SciPy 1.18.1）复算，幻灯片上的全部代码在这些版本上运行通过。

> All figures were redrawn from the slides' examples with Matplotlib 3.11; numbers were recomputed in Python (NumPy 2.5.4, SciPy 1.18.1), and every code cell on the slides runs on these versions.

考点速记 KEY POINTS

- 最常考：正规方程与 $A^T e = 0$。
  Most examined: the normal equation and $A^T e = 0$.
- 对数技巧要还原截距，并要求 $y > 0$。
  Log trick: undo the intercept; needs $y > 0$.
- 易错：`lstsq` 返回元组，`polyfit` 最高次在前。
  Traps: `lstsq` returns a tuple; `polyfit` is highest-first.
- 训练 SSE 不能用来选多项式阶数。
  Training SSE cannot choose the polynomial order.